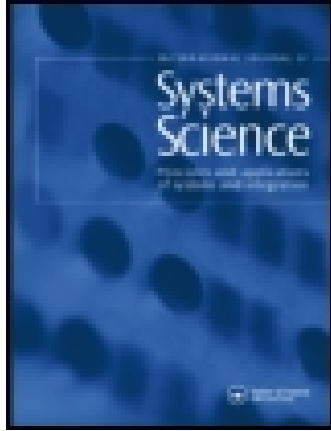


This article was downloaded by: [University of Sheffield]

On: 18 June 2014, At: 06:50

Publisher: Taylor & Francis

Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



International Journal of Systems Science

Publication details, including instructions for authors and subscription information:

<http://www.tandfonline.com/loi/tsys20>

An overview of population-based algorithms for multi-objective optimisation

Ioannis Giagkiozis^a, Robin C. Purshouse^a & Peter J. Fleming^a

^a Department of Automatic Control and Systems Engineering, University of Sheffield, Sheffield, S1 3JD, United Kingdom

Published online: 20 Aug 2013.



CrossMark

[Click for updates](#)

To cite this article: Ioannis Giagkiozis, Robin C. Purshouse & Peter J. Fleming (2013): An overview of population-based algorithms for multi-objective optimisation, International Journal of Systems Science, DOI: [10.1080/00207721.2013.823526](https://doi.org/10.1080/00207721.2013.823526)

To link to this article: <http://dx.doi.org/10.1080/00207721.2013.823526>

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms & Conditions of access and use can be found at <http://www.tandfonline.com/page/terms-and-conditions>

An overview of population-based algorithms for multi-objective optimisation

Ioannis Giagkiozis*, Robin C. Purshouse and Peter J. Fleming

Department of Automatic Control and Systems Engineering, University of Sheffield, Sheffield S1 3JD, United Kingdom

(Received 15 July 2012; final version received 5 June 2013)

In this work we present an overview of the most prominent population-based algorithms and the methodologies used to extend them to multiple objective problems. Although not exact in the mathematical sense, it has long been recognised that population-based multi-objective optimisation techniques for real-world applications are immensely valuable and versatile. These techniques are usually employed when exact optimisation methods are not easily applicable or simply when, due to sheer complexity, such techniques could potentially be very costly. Another advantage is that since a population of decision vectors is considered in each generation these algorithms are implicitly parallelisable and can generate an approximation of the entire Pareto front at each iteration. A critique of their capabilities is also provided.

Keywords: genetic algorithms; ant colony optimisation; particle swarm optimisation; differential evolution; artificial immune systems; estimation of distribution algorithms

1. Introduction

Population-based optimisation techniques (PBOTs) have received much attention in the past 30 years. This can be attributed to their inherent ability to escape local optima, their extensibility to multiple-objective problems (MOPs) and their ability to incorporate the handling of linear and non-linear inequality and equality constraints in a straightforward manner. They are applicable to a wide set of problems and impose very few constraints on the problem structure. For example, continuity and differentiability are not required. PBOTs also perform well on NP-hard¹ problems where exhaustive search is impractical or simply impossible. Since there is a population of decision vectors, the Pareto front (PF) can be approximated (theoretically) to arbitrary precision. Namely, assuming that the PF is continuous and that the selected algorithm is able to converge to the optimal points, by increasing the number of solutions the *resolution* of the PF approximation can be moderated at will.

An additional difficulty with MOPs, apart from the apparent increase in complexity, is that the notion of a solution is very different from the single-objective case. What does it mean then to say an algorithm *solves* an MOP? When dealing with single-objective problems, comparison of different objective function values is simple. If the task is to minimise an objective function, then solutions can directly be compared since scalars are totally ordered. A *solution* then is a decision vector that achieves this goal.

Assuming that the problem under consideration is being *solved* for practical purposes, that is, it is a real-world

problem and a design choice is to be made. Even if multiple competing objectives are under consideration, only one solution can usually be employed. This requires that a human expert or decision-maker (DM) to resolve the situation by selecting a small subset of the solutions presented by the algorithm. So, in this scenario, or more formally *a posteriori* preference articulation, the algorithm is endowed with the task of finding a set of alternative solutions. Subsequently, the DM will evaluate these solutions according to their preferences and make a selection. To facilitate this process, the algorithm guides a population of decision vectors toward a tradeoff surface, be it convex, concave or even discontinuous. This tradeoff surface should enable the DM to choose a solution that they believe would serve their purposes well. So, it becomes apparent that in MOPs a DM plays an integral role in the solution procedure. There are various paradigms of interaction involving the DM and the solution process, but in the present work *a posteriori* preference articulation is the main focus; for other types, see Miettinen (1999). This choice is based on the fact that this particular method of interaction with the DM allows a greater degree of separation between the algorithm and the decision-making process. Also this is invariably the reason as to why this paradigm is usually employed by researchers when developing new optimisation techniques: it enables the testing process to be conducted independently of the problem or application and most importantly it need not involve the DM at this stage. It should also be noted that we do not mention specific applications in this work. The only exception to this rule is when an application explicitly

*Corresponding author. Email: i.giakiozis@sheffield.ac.uk

results in creation of an algorithm highly regarded in the community. The reason for this decision is that this approach enables us to present a more concise conceptual view of PBOTs. A view that we hope would be helpful to the practitioner in search of an approach to solve a particular problem.

Other recent works that review population-based optimisation methods are due to Liu, Wang, Liu, and Wang (2011) and Zhou et al. (2011). Although the work of Liu et al. (2011) is not a survey, the authors present an interesting approach in unifying concepts employed in PBOTs, and as such contains useful reference material about previous attempts. Zhou et al. (2011) present a more thematic overview of multi-objective evolutionary algorithms (MOEAs) and also provide an extensive list of applications of evolutionary algorithms (EAs) in real-world problems. Although the work of Zhou et al. (2011) can serve as an excellent reference for researchers, there is an implicit assumption that the reader is familiar with certain concepts, and, as a result, it appears to be targeted towards experts in the field. Additionally, we have found no previous work that elaborates on individual features present in each algorithm family and provides a critique. We also include a section on available software that implements the algorithm families reviewed in this work.

The purpose of this work is to provide a general overview of the most prominent population-based multi-objective optimisation methodologies, serving as a map or a starting point in the search for a suitable technique, thus serving as a complement to existing works (Zlochin, Birattari, Meuleau, and Dorigo 2004; Dorigo and Blum 2005; Jin and Branke 2005; Reyes-Sierra and Coello 2006; Tapia and Coello 2007; AlRashidi and El-Hawary 2009; Das and Suganthan 2010; Liu et al. 2011; Zhou et al. 2011). As in cartography, maps are meant to contain important landmarks, but in order to enhance legibility a vast amount of detail has been abstracted or omitted. In that spirit, hybrid algorithms, for example, memetic algorithms (Moscato 1989), are not considered here, without any implications for the contribution they make. Additionally, although genetic programming (GP) instigated by Koza (1996) is based on the same principles as genetic algorithms (Holland 1975), it is also not considered here, since in the view of the authors, the main focus of GP is not multi-objective optimisation. Additionally, the internal representation of a solution in GP is usually a simple graph which is radically different from the representation of the methods we consider here, namely real, discrete or binary linear encoding.

The remainder of this paper is organised as follows. In Section 2, we portray a chronology of developments in population-based multi-objective optimisation algorithms and in Section 3 a generalisation of the functional principles of PBOTs is distilled. Subsequently, in Section 4, seven algorithm families are presented in their main form, namely as they apply to single-objective problems and in

the last subsection we elaborate on the ways that domain knowledge can be utilised to decide which of these methods would be more appropriate. As the methods used to extend the aforementioned algorithmic families to MOPs are often very similar, we discuss these in Section 5 in a more general context and discuss how they have been applied in Section 6. In Section 7, we comment on available software for implementing the methods discussed in this work and also provide a listing of the supported algorithm families. The paper concludes with Section 8 and Section 9. In Section 8, these methodologies are assessed against a set of criteria which describe a range of problem features. Section 9 promising research directions are discussed along with a summary and conclusion of this work.

2. Chronology

The origins of the inspiration to use a population of trial solutions in an optimisation algorithm can be traced back to Holland (1962) as a mixture of automata theory and theoretical genetics. Since then several algorithms have been developed that utilise a population of decision vectors and by means of applying variation and exploration operators this population evolves to *adapt* to the *environment*. The *environment*, represented by an evaluation (or objective) function, dictates which individuals in the population would be fit to enter the next generation. Holland (1975) formalised the idea presenting a more concrete framework for optimisation which he named genetic algorithms (GAs). The scheme quickly gained acceptance and Schaffer (1985) proposed an algorithm, the vector evaluated genetic algorithm (VEGA), based on Holland's GA to tackle multiple objectives simultaneously. The following year Farmer, Packard, and Perelson (1986) published a paper describing how knowledge about the human immune system can be used to create adaptive systems. This artificial immune system (AIS) shares many commonalities with the classifier system described by Holland (1975). Two years later, Goldberg (1989) introduced a conceptual algorithm of how Pareto dominance relations could be utilised to implement a multi-objective GA. This idea is heavily used to this day when extending single-objective algorithms to address MOPs.

Independently and in parallel with Holland, Rechenberg (1965) introduced evolution strategies (ESs) in the mid-1960s. The early versions of ESs had only two individuals in their population due to the limited computational resources available at the time. However, in subsequent years multi-membered versions of ESs were introduced by Rechenberg and further developed and formalised by Schwefel (1975).

Another important approach was introduced by Dorigo, Maniezzo, and Coloni (1991) which they called the ant system (AS). The AS, inspired by the ability of ants to find the shortest route from a food source to their nest, was used to solve combinatorial problems and almost a decade

later Dorigo and Di Caro (1999) presented a generalisation of the AS which they named ant colony optimisation (ACO). In the mid-1990s, Storn and Price (1995) brought forward a novel heuristic, differential evolution (DE). Its main characteristic was its conceptual simplicity and its wide applicability to a variety of problems with continuous decision variables. Later that same year, Eberhart and Kennedy (1995) introduced particle swarm optimisation (PSO), a new family of optimisers inspired by the flocking behaviour of pack animals. Almost a full circle is complete by the latest addition to PBOTs, namely estimation of distribution algorithms (EDAs) (Mühlenbein and Paass 1996; Larranaga and Lozano 2002). A full circle in the sense that concepts derived from nature were employed as inspiration for more than two decades, helping researchers solve complex problems. EDAs are one of the first metaheuristic algorithms not to rely on nature-inspired mechanisms to drive their search. Instead, EDAs depend on probabilistic models of better performing individuals in the population to generate the entire or part of the next generation.

3. General structure

A single-objective optimisation problem is defined as

$$\begin{aligned} & \min_{\mathbf{x}} f(\mathbf{x}) \\ \text{subject to } & \begin{cases} g_i(\mathbf{x}) \leq 0 & i = 1, \dots, m \\ h_i(\mathbf{x}) = 0 & i = 1, \dots, d \\ \mathbf{x} \in \text{dom } f & \text{dom } f \subset \mathbb{R}^n \end{cases} \end{aligned} \quad (1)$$

where m and d are the number of inequality and equality constraints, respectively, n is the size of the decision vector $\mathbf{x}$ and $\text{dom } f$ is the domain of definition of the objective function. The type of the functions f , g and h as well as the domain of f are the key factors that determine the type of the problem defined by (1). For example, when the objective function is convex and is defined over a convex domain, and the inequality constraints are convex functions, while the equality constraints are affine, then (1) is a convex optimisation problem (Boyd and Vandenberghe 2004). An optimisation problem defined by (1) is considered to be solved when a decision vector $\tilde{\mathbf{x}} \in \text{dom } f$ is found that

$$\begin{aligned} & f(\tilde{\mathbf{x}}) \leq f(\mathbf{x}), \text{ for all } \mathbf{x} \in \text{dom } f, \\ \text{and } & \begin{cases} g_i(\tilde{\mathbf{x}}) \leq 0 & i = 1, \dots, m \\ h_i(\tilde{\mathbf{x}}) = 0 & i = 1, \dots, d. \end{cases} \end{aligned} \quad (2)$$

Such a decision vector, $\tilde{\mathbf{x}}$, is said to be a *global minimum*. A local minimum is defined as

$$\begin{aligned} & f(\tilde{\mathbf{x}}) \leq f(\mathbf{x}), \text{ for all } \mathbf{x} \in \{\mathbf{x} : \|\mathbf{x} - \tilde{\mathbf{x}}\|_2 \\ & \leq r, r \in \mathbb{R}, \mathbf{x}_c \in \text{dom } f\}, \\ \text{and } & \begin{cases} g_i(\tilde{\mathbf{x}}) \leq 0 & i = 1, \dots, m \\ h_i(\tilde{\mathbf{x}}) = 0 & i = 1, \dots, d, \end{cases} \end{aligned} \quad (3)$$

and is usually what can be found for non-convex problems, since it is usually impracticable to search the entire feasible set so as to ensure that the global minimum is found.

A useful device is the definition of the *feasible set* in decision space, namely

$$\begin{aligned} S = \{ \mathbf{x} : \mathbf{x} \in \text{dom } f, g_i(\tilde{\mathbf{x}}) \leq 0, i = 1, \dots, m, \\ h_i(\tilde{\mathbf{x}}) = 0, i = 1, \dots, d \}, \end{aligned} \quad (4)$$

which is also called the *feasible region*. With the help of (4), (1) can be rewritten in a more compact form,

$$\begin{aligned} & \min_{\mathbf{x}} f(\mathbf{x}) \\ \text{subject to } & \mathbf{x} \in S. \end{aligned} \quad (5)$$

Population-based metaheuristics operate on a set of decision vectors, termed *population*, which in this work we denote by $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, where $\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,n})$, N is the number of decision vectors in the *population* (or *size* of the population) and n is the number of decision variables in each decision vector. It should also be mentioned that decision vectors are in some contexts referred to as *individuals*, this is mostly to emphasise the nature-derived principles used in various algorithms and is employed to promote conceptual coherency rather than functional discrimination.

PBOTS, despite the diversity in their problem-solving approach, share more similarities than differences. In general they are comprised of five parts (see Figure 1): the main algorithm, an extension to tackle MOPs (discussed in Section 5), an extension to deal with constrained optimisation problems, a part to maintain promising solutions and a part to halt the algorithm execution based usually on some notion of convergence. The main algorithm deals primarily with single-objective optimisation, and is comprised of three parts. Namely, a component that combines information within the population (recombination), one perturbing some individuals to enhance search space exploration (mutation) and a component to select promising individuals to be part of the new generation (selection). In Algorithm 1, mutation and recombination are lumped into a single operator termed *variation* to maintain generality. By recombination we mean the superset of operators, usually crossover,

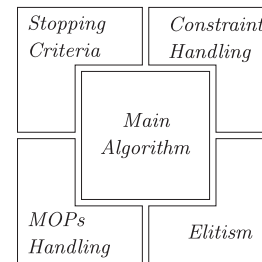


Figure 1. PBOT components.

that utilise two or more decision vectors to form one or more new decision vectors. All three parts of the main algorithm have some deterministic and stochastic features. If these three operations are chosen correctly, the population $\mathbf{X}^{(G+1)}$ will have a better chance of being *superior* to the previous generation $\mathbf{X}^{(G)}$. Superiority in this context is determined in different ways depending on the type of the objective function; for example, if it is a scalar objective function, then for a minimisation problem a superior solution is one that evaluates to a smaller number compared to its predecessor.² In the case of an MOP, this issue is more ambiguous (see Section 5). It should be noted, however, that there is an implicit assumption here. Namely, that there is some information in the parent population which can be exploited to *construct* better solutions in the offspring population. Should this assumption fail so will the algorithm in identifying optimal, or near-optimal, solutions.

Algorithm 1 PBOT conceptual algorithm

- 1: Initialise population
 - 2: Evaluate the objective function
 - 3: **repeat**
 - 4: Evaluate the population quality
 - 5: Apply the variation operator
 - 6: Evaluate the objective function
 - 7: **until** termination condition is met
-

The general algorithm in PBOTs can be seen in Algorithm 1; the population is usually initialised within the feasible region, if possible, and uniformly distributed in the absence of prior information. Most population-based metaheuristics are based on empirical knowledge and techniques that would steer the search toward desirable regions of S or directly replicate mechanisms found in nature.

4. Algorithm families

The main algorithm, at least as was initially introduced, of the seven algorithm families is presented in this section. The motivation for this short introduction is to provide context for our discussion in Sections 4.8, 5 and 8.

4.1. Genetic algorithms

GAs are a family of stochastic metaheuristic search algorithms with links to cybernetics (Holland 1962; Goldberg and Holland 1988) and philosophically connected to Darwin's theory of evolution (Darwin 1858). Social dynamics are simulated based on the premise that the *survival of the fittest* paradigm does produce qualitatively better *individuals* with the passage of several generations. Individuals are in effect decision vectors whose fitness is evaluated using the objective function. The representation of decision vectors in the basic GA is a binary *string* termed *chromosome* comprising of *genes* (decision variables). Each chro-

sosome is mapped to a *phenotype*. A phenotype is a representation that the objective function can directly utilise to evaluate the fitness of an individual. The main operators

Algorithm 2 Simple genetic algorithm

- 1: Initialise the population
 - 2: **repeat**
 - 3: Evaluate the objective function
 - 4: Apply crossover and mutation
 - 5: Select solutions for the next population
 - 6: **until** termination condition is met
-

used in GAs are crossover and mutation. These two operators are combined in a fashion to balance exploitation and exploration of the search space. Crossover and mutation, see Figure 2, are applied to the chromosomes, thus facilitating the creation of an abstraction layer with respect to the phenotype. This layer of abstraction insulates the internal implementation of GAs from the problem structure, specifically the representation type of the decision variables. The main operators in GAs have been extended to deal directly with continuous decision variables (Deb and Agrawal 1994; Voigt, Mühlenbein, and Cvetkovic 1995; Ono and Kobayashi 1997) which is useful when full machine precision is required and all decision variables are continuous.

4.2. Evolution strategies

The introduction of ESs is due to Rechenberg (1965). This early version of ES was a two-membered mutation-based algorithm, the so-called (1 + 1)-ES. Although ESs share similar features and inspirational background to GAs, there are two distinct differences. First, ESs do not employ a crossover operator, and second, ESs utilise real decision variables as opposed to GAs that originally employed binary encoded *strings*. However, these differences have blurred over time due to developments on both sides. Effectively, both methodologies pursued adaptation and were inspired by the cybernetic movement of that era.

There is a general consensus that both mutation and recombination operators perform an important role and

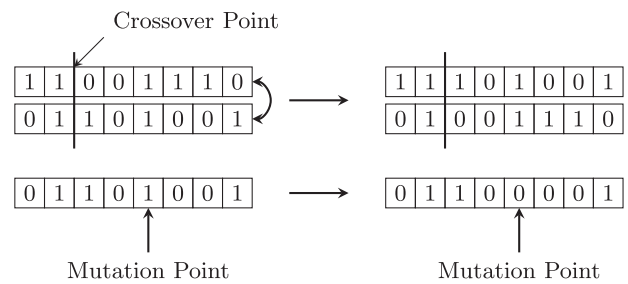


Figure 2. Single-point crossover and bit-flip mutation operator in GAs.

eventually both were employed to various degrees in these two approaches. The main argument concerns their effect on the evolving population. Holland, with his famous building block hypothesis (Holland 1975), promoted the idea that the recombination operator had a positive influence on the evolving population due to short *schemas* with good *fitness* recombining to form even better individuals. Beyer argued that this was not the case; his suggestion was that the crossover operator had the effect of *genetic repair* (Beyer 1997). Beyer's hypothesis was that the features that *survive* the crossover operator are the ones common to both parents and not the small *schemas* with desirable elements.

As previously mentioned, the first version of ES was $(1 + 1)$ -ES. This notation refers to the fact that there are two members in the population, namely the *parent* and one *offspring*, and the selection for the *surviving* individual is performed among the two. The $+$ sign essentially refers to the selection mechanism. Some later versions of ES use the comma notation, denoting that the *parent* takes no part in the selection process and new individuals are selected only from the offspring pool. The main *search* operator in $(1 + 1)$ -ES is the mutation operator defined as follows:

$$\mathbf{x}^{(G+1)} = \mathbf{x}^{(G)} + \mathcal{N}(\mathbf{0}, \sigma), \quad (6)$$

where $\mathcal{N}(\mathbf{0}, \sigma)$ is a vector of random numbers of size n drawn from the normal distribution with zero mean and σ standard deviation. Here $\mathbf{x}^{(G)}$ represents the parent solution and $\mathbf{x}^{(G+1)}$ the offspring. If $f(\mathbf{x}^{(G+1)}) \leq f(\mathbf{x}^{(G)})$, then the offspring is retained and the parent is discarded. Otherwise, the same parent is used in the next generation.

ESs were extended to a *true* population-based optimisation methodology by Schwefel (1981). Schwefel introduced $(\mu + \lambda)$ -ES as well as (μ, λ) -ES. Here μ is the number of parents producing λ offspring. The resulting population, after the selection process, is reduced to the number of parents μ . Schwefel combined these two variants to enable a parallel implementation of ESs as well as to experiment with adaptation in the control parameters, such as σ (Bäck, Hoffmeister, and Schwefel 1991). For further details on the $(\mu + \lambda)$ -ES, the reader is referred to Schwefel (1981), Bäck et al. (1991) and Beyer and Schwefel (2002).

4.3. Artificial immune systems

The immune system (IS) is a fine-tuned concert of a vast array of cells acting in synchronism in order to protect the body from pathogens. Pathogens are substances or microorganisms that can affect the balance of the body (*homeostasis*), impairing its normal functioning and even cause death. The IS responds, *immune response*, to such threats, creating specialised cells capable of neutralising pathogens and restoring balance. In order for this procedure to progress smoothly, the IS must attack the source of the disturbance while minimising the effect this action has on healthy cells

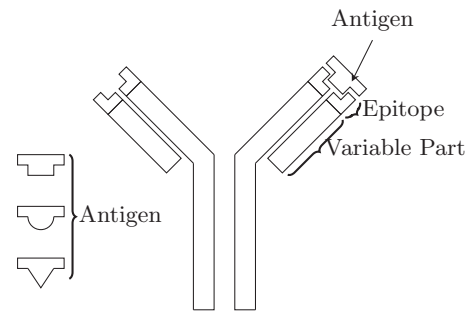


Figure 3. Antibody schematic diagram.

in the body – *specificity*. Once a particular threat has been dealt with, the second time the body is exposed to the same pathogen the immune response will be swifter and stronger to such a degree that the organism might not even realise that it had been afflicted by a pathogen, so the IS also exhibits *memory*.

Pathogens have certain features that the IS can identify. These features are called *antigens*, and their presence in the organism is detected by *antibodies*. Antibodies are highly specialised molecules that have the capacity to identify specific antigens. An antibody, see Figure 3, identifies an antigen by binding on it, thus serving as a beacon signalling other cells in the IS to devour the intruder. Although antibodies are highly specialised, their diversity is enormous, thus enabling the IS to attack a vast variety of antigens. When a particular antigen is identified, the antibody that successfully identified it will proliferate by cloning via a process termed the *clonal selection principle* (Mohler, Bruni, and Gandolfi 1980; Dasgupta and Forrest 1999). Once the pathogens have been eliminated, superfluous antigens are dissolved mainly via the process of *apoptosis* (Krammer 2000). However, some cells responsible for the proliferation of this particular type of antibodies survive as memory cells (Parijs and Abbas 1998), and if the host is infected again by the same pathogen, these memory cells will be activated to produce antibodies much faster compared with the first encounter, *primary response*, with this particular antigen.

Although this introduction to the IS will serve the purpose of illustrating the parallelism of the biological system with AISs, it does not even scratch the surface of the delicate intricacies present in the IS. The interested reader is referred to Parham and Janeway (2005).

AISs have been employed in various applications, such as intrusion detections systems, virus detection, anomaly detection fault diagnosis and pattern recognition (Dasgupta and Attoh-Okine 1997). AISs have been used in place of fitness sharing techniques in GAs to maintain diversity and distribute the population evenly along the PF (Coello and Cortés 2005). Another interesting use is in time-dependent optimisation problems where AISs successfully *adapt* to a changing problem (Gasper and Collard 1999). Despite

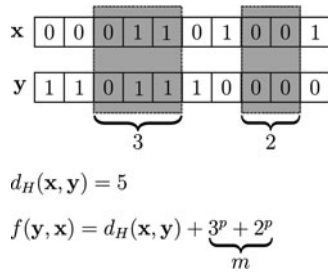


Figure 4. Similarity measure used to simulate the clonal selection principle (Coello and Cortés 2002). Here, x and y represent an antigen and antibody, respectively. The similarity measure, $f(\cdot, \cdot)$, is used to modulate the mutation strength in an inverse proportional manner.

these applications, direct use of AISs to optimisation problems prior to 1999 is scarce. This could potentially be attributed to the complexity involved in implementing an AIS.

Although there is no single uniform framework that describes AISs for optimisation, a representative algorithm based on the principles of ISs is due to Coello and Cortés (2002). Here we describe the main features of this method, while for further details the reader is referred to Coello and Cortés (2002, 2005). Coello and Cortés (2002) draw upon the clonal selection principle by labelling elite individuals as antigens; these are solutions that are non-dominated and feasible. Inferior individuals were labelled as *antibodies*, namely solutions that are either infeasible, dominated or both. Subsequently, a modified Hamming distance metric was employed for fitness assignment,

$$f(x, y) = d_H(x, y) + m, \quad (7)$$

where m is an additional biasing factor that depends on the number of similar groups found between x and y . An illustration of the calculations involved in evaluating (7) as well as a definition of m is given in Figure 4. Subsequently, the mutation strength for the generation of new individuals is determined by a reverse proportional relationship with (7) (Coello and Cortés 2002). The aforementioned algorithm cannot be classified as a modified genetic algorithm as new individuals are formed only by means of mutation. In this respect, the method presented by Coello and Cortés (2002) is similar to ESs.

4.4. Ant colony optimisation

Ants come from the same family as wasps and bees (Choe and Perlman 1997), namely *Hymenoptera formicidae*. As individuals, ants' capabilities are limited. Apart from their extraordinary strength their sight is very limited and their hearing and sense of smell depend on their antennae. Despite these facts, ants as a collective exhibit very interesting behaviour patterns (Deneubourg, Aron, Goss, and Pasteels

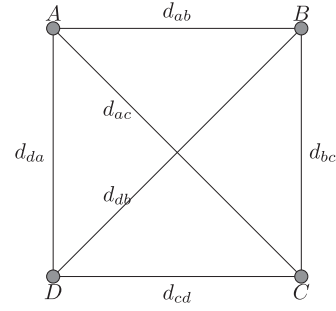


Figure 5. TSP with four cities. Although the edges connecting the cities seem equidistant, they need not be. The distance of two cities is represented by d .

1990). In their search for food, ants explore their nest surroundings in a random-walk pattern. However, when an ant discovers food, it returns to the nest releasing pheromones (Deneubourg et al. 1990) of varying intensity depending on the quality and quantity of the discovered food source. Other ants close to the pheromone trail are more likely to follow that path and, if there is still food to be found at the end of the trail, they release pheromones as well, thus reinforcing the pheromone scent which in turn increases the probability that more ants will follow that path. Alternatively, when an ant reaches the destination indicated by the pheromone trail and does not discover anything interesting, no pheromone is released and progressively the path becomes less and less attractive as pheromones evaporate.

Dorigo et al. (1991) introduced an algorithm, *ant system*, inspired by the emergent behaviour of ants. The AS was tailored to solve combinatorial problems and the travelling salesman problem (TSP) was used as a test bed (see Figure 5). Almost a decade later Dorigo and Di Caro (1999) formalised the AS and other similar methods in a unified framework, termed ant colony optimisation. ACO is comprised of N individuals called *ants*. These are distributed among μ cities and each ant progressively creates a candidate solution for the problem. The ants *decide* which city they will next visit based on the pheromone trail (τ) associated with each edge leading to a particular city and the separating distance of the cities. Assuming that an ant starts off from city A in Figure 5, it has three options, namely to go to B , C or D . Each of these edges has a pheromone trail associated with it. The probabilistic transition rules are controlled by two parameters, namely τ and η . The parameter τ can be initialised using prior information, if available, or with the same value for all edges. Initially η represented the *visibility* of the ants which depended on the distance of a city from the next and was defined as $\eta_{AB} = \frac{1}{d_{AB}}$ for the edge connecting city A with B . The smaller the η_{AB} , the larger the distance d_{AB} and the less likely that an ant in city A chooses city B for its next step in the tour (see Equation (9)). As ants *traverse* across the edges constructing

different solutions, the pheromone across the edges is updated as

$$\tau_{i,j} = (1 - \rho)\tau_{ij} + \sum_{s=1}^N \Delta\tau_{i,j}^s, \quad (8)$$

where ρ is defined as the evaporation rate of the pheromones and $\Delta\tau_{i,j}^s$ is the amount of pheromone added by ant s on the edge connecting the i th and j th node. Therefore, the larger the deposit of pheromones across an edge i, j , the larger the $\tau_{i,j}$; hence, the probability that more ants will select this path is increased. The probabilistic transition rule, based on τ and η , that governs the *behaviour* of ants is summarised by the following relation:

$$p_{i,j} = \begin{cases} \frac{\tau_{i,j}^\alpha \eta_{i,j}^\beta}{\sum_{m=1}^{\mu} \tau_{i,m}^\alpha \eta_{i,m}^\beta} & \text{if } j \text{ is a valid destination,} \\ 0 & \text{if } j \text{ is not a valid destination,} \end{cases} \quad (9)$$

where a valid destination for an ant is a node that the particular ant is allowed to visit at that particular stage. The probability that an ant on node i will visit node j is given by $p_{i,j}$, and μ is the number of nodes in the problem. For the TSP problem a node is a *valid* destination if a particular ant has not visited that node. Also α and β are real and positive numbers used to favour either τ or η ; if $\alpha = \beta$, the information gained from τ is considered equally important with η .

The general ACO algorithm has the following stages:

Step 1: Initialise pheromone values τ for all the edges. Usually all the edges receive the same pheromone value at the start of the algorithm, unless prior information is available indicating that favouring certain edges would increase the convergence speed.

Step 2: Construct a solution for each ant, $\mathbf{x}_s$.

Step 3: Update the pheromone values for each edge.

Step 4 Go to **Step 2** until stopping criteria are met.

The rules for constructing a solution $\mathbf{x}_s$ for each ant are problem-dependent.

Since 1991, ACO has been successfully applied to various combinatorial problems such as vehicle routing, sequential ordering, project scheduling, multiple knapsack and protein folding (Dorigo, Birattari, and Stutzle 2006).

4.5. Differential evolution

DE was introduced by Storn and Price (1995) as a global search method over real decision vectors, although later extended versions for integer (Lampinen and Zelinka 1999) and binary representations (Pampara, Engelbrecht, and Franken 2006) were introduced. The major strengths of DE

are its conceptual simplicity, ease of use and implementation; also the number of tuning parameters is very small, and the same parameter values with no or very little change are found to be applicable to a wide range of problems.

For a population of size N and decision vectors of dimension n , the current generation is

$$\mathbf{x}_i^{(G)} \text{ for } i = \{1, 2, \dots, N\}. \quad (10)$$

The basic DE algorithm has three stages in its iteration phase: mutation, crossover and selection. During the mutation stage, all decision vectors in the population are perturbed, resulting in a temporary new population $\mathbf{V}^{(G+1)}$ of the same size with $\mathbf{X}^{(G)}$. The mutation operator in DE is described as

$$\begin{aligned} \forall \mathbf{x}_i^{(G)}, i = \{1, 2, \dots, N\}, \\ \mathbf{v}_i^{(G+1)} = \mathbf{x}_{r_1}^{(G)} + F(\mathbf{x}_{r_2}^{(G)} - \mathbf{x}_{r_3}^{(G)}), \end{aligned} \quad (11)$$

and if $f(\mathbf{v}_i^{(G+1)}) < f(\mathbf{x}_i^{(G)})$ the newly formed parameter vector $\mathbf{v}_i^{(G+1)}$ is assigned to $\mathbf{x}_i^{(G+1)}$; otherwise, $\mathbf{x}_i^{(G)}$ is retained in the next generation. The parameters $r_1, r_2, r_3 \in \{1, 2, \dots, N\}$ are sampled at random without replacement from the set $\{1, \dots, N\} - \{i\}$ for each individual in the population. The parameter $F \in [0, 2]$ is a scaling factor controlling the variation ($\mathbf{x}_{r_2}^{(G)} - \mathbf{x}_{r_3}^{(G)}$). An illustration of the mutation operator is shown in Figure 6.

The crossover operator in DE is defined as

$$\begin{aligned} \mathbf{u}_i^{(G+1)} &= \left(u_{i,1}^{(G+1)}, u_{i,2}^{(G+1)}, \dots, u_{i,n}^{(G+1)} \right), \\ &\text{where} \\ u_{i,j}^{(G+1)} &= \begin{cases} v_{i,j}^{(G+1)} & \mathcal{U}(0, 1) \leq C_r \text{ or } j = \text{ridx}(i), \\ x_{i,j}^{(G)} & \mathcal{U}(0, 1) > C_r \text{ and } j \neq \text{ridx}(i), \end{cases} \quad (12) \\ &j = 1, 2, \dots, n, \end{aligned}$$

and C_r is the crossover rate parameter, bounded in the range $[0, 1]$, $\text{ridx}(i)$ is a random index in the range $\{1, \dots, n\}$,³ and $\mathcal{U}(0, 1)$ is a random number sampled from a uniform distribution in the domain $[0, 1]$. Once the new decision vectors $\mathbf{u}_i^{(G+1)}$ have been generated, the selection is based on the *greedy* criterion, which can be stated as

$$\mathbf{x}_i^{(G+1)} = \begin{cases} \mathbf{u}_i^{(G+1)} & \text{if } f(\mathbf{u}_i^{(G+1)}) < f(\mathbf{x}_i^{(G)}) \\ \mathbf{x}_i^{(G)} & \text{otherwise.} \end{cases} \quad (13)$$

So the DE algorithm progresses exactly as Algorithm 2 but using the crossover, mutation and selection operators defined above, in place of the variation operator. Some general guidelines on tuning DE can be found in Storn (1996).

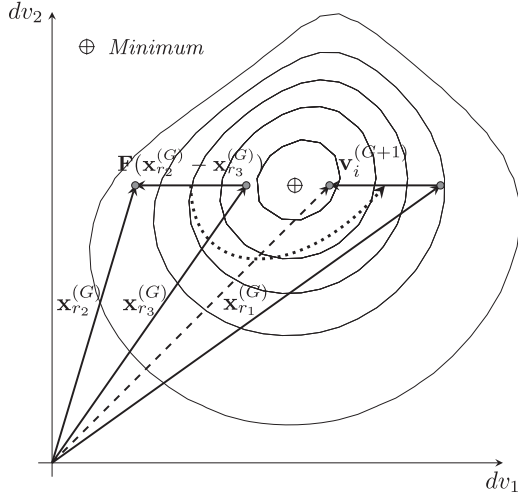


Figure 6. Illustration of the mutation operator in DE; in this case, $F = 1$ and $\mathbf{x}_i^{(G+1)} = \mathbf{v}_i^{(G+1)}$ since for this instance $f(\mathbf{x}_i^{(G+1)}) < f(\mathbf{x}_i^{(G)})$, and, dv_1 and dv_2 stand for decision variable 1 and 2, respectively.

4.6. Particle swarm optimisation

PSO, introduced in Eberhart and Kennedy (1995) and Kennedy and Eberhart (1995), was inspired by the flocking behaviour of birds and swarm theory. In PSO, as observed in nature, each agent has a rather limited repertoire of behaviours, while the collective exhibits complex expressions. In the initial PSO algorithm, the particles (decision vectors) use a simple update rule to update the velocity and, consecutively, the position of each particle. An archive is maintained that contains the best achieved objective function values for each particle,

$$\mathbf{P} = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}, \quad (14)$$

so $f(\mathbf{p}_i) \leq f(\mathbf{x}_i^{(G)})$ and if $f(\mathbf{p}_i) > f(\mathbf{x}_i^{(G+1)})$, then $\mathbf{p}_i = \mathbf{x}_i^{(G+1)}$, and so on. Additionally, a global best position is maintained, $\mathbf{p}_g$, for which the following condition must hold: $f(\mathbf{p}_g) \leq f(\mathbf{x}_i^{(G+1)})$, for all $i \in \{1, 2, \dots, N\}$. If this is not true, $\mathbf{p}_g$ is updated using the following rule: $\mathbf{p}_g = \{\mathbf{x}_i : \min_i f(\mathbf{x}_i)\}$. The velocity update rule is

$$\begin{aligned} \mathbf{v}_i^{(G+1)} = & \mathbf{v}_i^{(G)} + c_1 \mathcal{U}(0, 1) (\mathbf{p}_i - \mathbf{x}_i^{(G)}) \\ & + c_2 \mathcal{U}(0, 1) (\mathbf{p}_g - \mathbf{x}_i^{(G)}), \end{aligned} \quad (15)$$

where c_1 and c_2 are positive constants and $\mathcal{U}(0, 1)$ is a random number in the range $[0, 1]$. A suggested value for c_1 and c_2 , for problems when no prior information is available, is that both are set to 2 (Kennedy and Eberhart 1995). This would effectively result in a multiplier with a mean value of 1, thus balancing in the mean the bias toward the points $\mathbf{p}_g$ and $\mathbf{p}_i$ for each particle (Shi and Eberhart 1998). A

modification to (15) was presented by Shi and Eberhart (1998) introducing a multiplying factor w to $\mathbf{v}_i^{(G)}$, the *inertia weight*, resulting in the following velocity update relation:

$$\begin{aligned} \mathbf{v}_i^{(G+1)} = & w \cdot \mathbf{v}_i^{(G)} + c_1 \mathcal{U}(0, 1) (\mathbf{p}_i - \mathbf{x}_i^{(G)}) \\ & + c_2 \mathcal{U}(0, 1) (\mathbf{p}_g - \mathbf{x}_i^{(G)}), \end{aligned} \quad (16)$$

where w can be constant, a function of the current generation G or even a function of a metric measuring the convergence of the algorithm. (For example, the normalised hypervolume indicator could be utilised.) A value of 1 for w results in the regular velocity update rule, while a value below 1 progressively decreases the average velocity of the particles, biasing the search to regions local to the particles. Alternatively, a value above 1 leads to a progressive increase in the velocity of particles, resulting in a more explorative behaviour. Subsequently, the position of the new particles is calculated in the following way (see Figure 7):

$$\mathbf{x}_i^{(G+1)} = \mathbf{x}_i^{(G)} + \mathbf{v}_i^{(G+1)}. \quad (17)$$

PSO does not have an explicit selection operator, although the archive $\mathbf{p}$ could qualify as such.

Several variants of the described PSO algorithm have been devised; the interested reader is referred to Tripathi, Bandyopadhyay, and Pal (2007) and del Valle, Venayagamoorthy, Mohagheghi, Hernandez, and Harley (2008). Other interesting methods that cannot be classified as particle swarm optimisers but inherit some features are the group search algorithm (He, Wu, and Saunders 2009) and several variants of the bee colony optimisation (Akay and Karaboga 2012; Karaboga, Ozturk, Karaboga, and Gorkemli 2012).

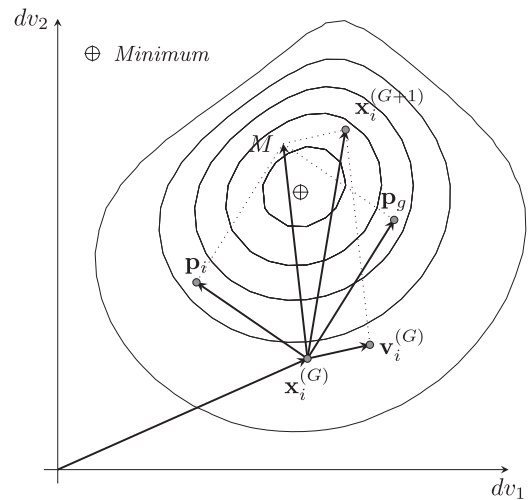


Figure 7. Particle update illustration in PSO, where $M = c_1 \mathcal{U}(0, 1) (\mathbf{p}_i - \mathbf{x}_i^{(G)}) + c_2 \mathcal{U}(0, 1) (\mathbf{p}_g - \mathbf{x}_i^{(G)})$ and $c_1 \mathcal{U}(0, 1)$, $c_2 \mathcal{U}(0, 1) = 1$ for clarity. As before, dv_1 and dv_2 stand for decision variables 1 and 2, respectively.

4.7. Estimation of distribution algorithms

In 1995, several crossover operators in GAs appeared that used probability distributions, for example, simulated binary crossovers (Deb and Agrawal 1994), unimodal normal distribution crossovers (Ono and Kobayashi 1997) and fuzzy recombination (Voigt et al. 1995), to name but a few. These crossover operators, although they did not recombine more than two or three individuals from the population, did use a probability distribution that was based on the parent solutions to generate the offspring. Estimation of distribution algorithms can, in a way, be seen as an expansion to the idea behind these crossover operators. The generalisation is straightforward. Instead of using a crossover and mutation operator in the GA, a probability distribution over the most prominent of solutions can be estimated and then utilised to produce new individuals in the population (Mühlenbein and Paass 1996). Arguably, another source of inspiration in the creation of EDAs has come from the statistics community; for example, the cross entropy method (Rubinstein 1999, 2005) was initially used for rare event estimation and probability collectives (Bieniawski, Wolpert, and Kroo 2004) have a game theoretic foundation (Bieniawski 2005; Wolpert, Strauss, and Rajnarayan 2006).

A typical EDA, see Algorithm 3, proceeds very similarly to a GA. The difference is that on every generation a subset of the population, usually the *better* half of the population, is selected and based on that subset a probabilistic model is created. Then this model is sampled and the resulting new individuals are merged with the old population while maintaining the population size constant.

Algorithm 3 Estimation of the distribution algorithm

- 1: Initialise the population $\mathbf{X}^{(0)}$
 - 2: Evaluate the objective function
 - 3: **repeat**
 - 4: Select promising solutions $\mathbf{X}_p$ from $\mathbf{X}^{(G)}$
 - 5: Create a probabilistic model P based on $\mathbf{X}_p$
 - 6: Generate new solutions $\mathbf{X}_n$ by sampling P
 - 7: Evaluate the objective function for $\mathbf{X}_n$
 - 8: Combine $\mathbf{X}_n$ and $\mathbf{X}^{(G)}$ to create $\mathbf{X}^{(G+1)}$
 - 9: **until** the termination condition is met
-

While EDAs have successfully been applied to a diverse problem set, involving real and discrete decision variables, outperforming rival algorithms (Hauschild and Pelikan 2011), it is not so trivial to generate the required probabilistic model. Also, the model type strongly depends on the decision variable type, whether or not the decision variables are coupled, to what extent and in what way. Assuming that all decision variables are independent, while in fact there are multiple interdependencies, would grossly mislead the algorithm (Pelikan, Goldberg, and Lobo 2002). Additionally, these challenges increase in difficulty when dealing with MOPs (Laumanns and Ocenasek 2002; Sastry,

Goldberg, and Pelikan 2005). Despite these difficulties, further research in EDAs seems promising due to their inherent adaptive nature that enables them to scale well compared with other algorithms for some problems (Shah and Reed 2011). Another strong point is that prior knowledge can effectively be exploited by biasing directly the initial population (Sastry 2001) or by biasing the model creation procedure (Mühlenbein and Mahnig 2002). However as seen in Figure 14 the research activity in EDAs is still in relatively low levels, a fact that can be ascribed to the difficulty of creating a probabilistic model capable of capturing dependencies in a way that does not substitute one difficult problem, namely the original optimisation problem, with another, that of building a probabilistic model.

4.8. So why not a single approach?

As is seen in Figure 14, the number of publications appearing per year is constantly increasing for all the above algorithm families, albeit not at the same pace. However, given their similarities and according to the *survival of the fittest* principle, one would expect some methods to fade away. This seems not to be the case for the above seven algorithm families, which is one of the reasons we elected to present them in this work.

This *survival* must mean that there is a niche that every family fulfils better than the others, while no algorithm family is *dominated*⁴ by any other in terms of performance. This result is not surprising and is partially explained in Wolpert and Macready (1997). However, that niche seems to be rarely expressed, with a few exceptions of course; for example, ACO are renowned for their applications on combinatorial problems (see Figure 15), but what of the others? At this point we exploit what Ullman (2009) considers to be a bad practice in research, namely for researchers to explore the unsolved problems reading the conclusions of published articles and not focusing their work on the problems that they can solve. If this is common practice, which Michalewicz (2012) seems to agree with, then we will be able, to some extent, to identify the strengths and weaknesses of the aforementioned algorithm families by the number of papers published with respect to different problems (see Figure 15).

Consider convex optimisation algorithms. In this class, gradient information is exploited to determine a direction of search. However, real-world problems do not always conform to the structure that such a class of optimisation methods can attack. One way that a problem can fail the convexity test is illustrated in Figure 8. This type of problems are called multi-modals and have multiple local optima. In this setting, a particle swarm optimisation algorithm would very well suit as it is conceivable that in the depicted scenario in Figure 8 a PSO-based algorithm has the potential to overcome the ridges and locate eventually the global optimum, $\tilde{x}$. However, the fact that the swarm of solutions

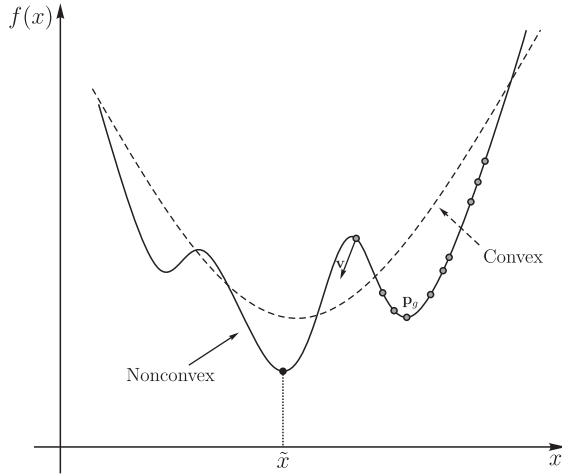


Figure 8. Illustration of a convex (dashed line) and non-convex (continuous line) multi-modal function. The points on the non-convex function represent a *swarm* of solutions in a PSO algorithm.

in Figure 8 has reached to that location is due to their following the solution $\mathbf{p}_g$ which in turn follows a path that is similar to that of a gradient descent algorithm. For this problem an algorithm with more *aggressive* stochastic operators would require more function evaluations compared to PSO. Problems with this structure are prevalent in evolutionary algorithm test functions (see e.g. Deb et al. 2002; Huband et al. 2006; Saxena et al. 2011). For the same reasons, differential evolution would have the potential to perform well in such a class of problems.

Now consider a different type of non-convex function (see Figure 9). In this case the *ridge* would be more difficult to *cross* using PSO; however, it would be much easier for an algorithm whose stochastic component is more dominant,

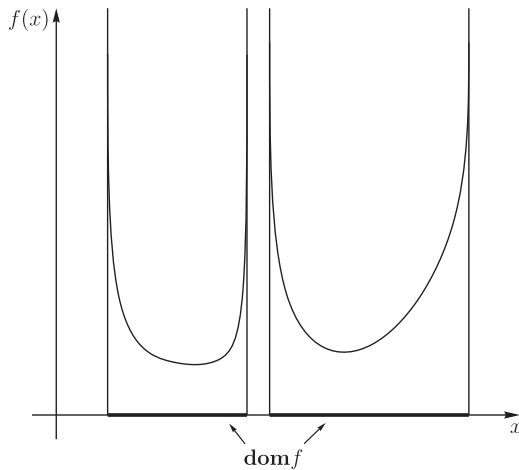


Figure 9. A discontinuous objective function. The minimum is located in the left lobe however algorithms with *weak* stochastic components that most of their initial population is located in the right lobe will have difficulties in locating the global optimum.

for example, ESs or EDAs. Of course, given the number of crossover operators available for GAs, it could be argued that such a behaviour could be simulated by judicious selection of such operators. Moreover, we neglected the fact that all the aforementioned methods have tunable parameters, which in turn alter their *behaviour*. Nevertheless, the different conceptual paradigms may enable the practitioner select these parameters more easily for a certain type of algorithms when applied to a problem with a structure that *favours* that particular algorithm family.

5. Multiple-objective problems

When the objective function is vector valued, that is, $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^k$, then the optimisation problem becomes more complex. This complexity stems from the fact that now there exists the possibility that there is no single-objective function value $\mathbf{F}(\tilde{\mathbf{x}}) \leq \mathbf{F}(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^n$, as was the case for single-objective problems. Therefore, in all but the most trivial case where all the scalar objective functions are *harmonious*, namely when all the objectives are positively correlated (Purshouse and Fleming 2003), only a partial ordering can be induced without the preference structure of the DM.

5.1. Problem setting

A multiple-objective minimisation problem can be defined as follows:

$$\begin{aligned} \min_{\mathbf{x}} \mathbf{F}(\mathbf{x}) &= (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_k(\mathbf{x})) \\ \text{subject to } \mathbf{x} &\in S, \end{aligned} \quad (18)$$

where k is the number of objective functions, S is the feasible set in decision space, $\mathbf{x}$ is the decision vector and $f_i(\mathbf{x})$ are scalar objective functions. Let $\mathbf{F} : S \rightarrow Z$, namely the forward image of the objective function, then the set Z is a feasible set in objective space. The $\min_{\mathbf{x}} \mathbf{F}(\mathbf{x})$ notation is interpreted as: minimise the vector valued function $\mathbf{F}(\mathbf{x})$ over all $\mathbf{x} \in S$ and should not be confused with the $\min$ operator which returns the minimum element of a set. It should also be noted that this definition could be used to describe a constrained or unconstrained minimisation problem depending on how S is defined (Equation (5)). Further the fact that minimisation is assumed is not restrictive because the problem of maximising $-f$ is equivalent to the problem of minimising f and vice versa. In the special case where $k = 1$, (18) becomes a single-objective minimisation problem. It is implicitly assumed that the scalar objective functions are mutually competing and perhaps are incommensurable while the goal is to minimise all of them simultaneously. If this is not the case, then no special treatment is needed since minimising one of the scalar objective functions automatically results in minimisation of the rest.

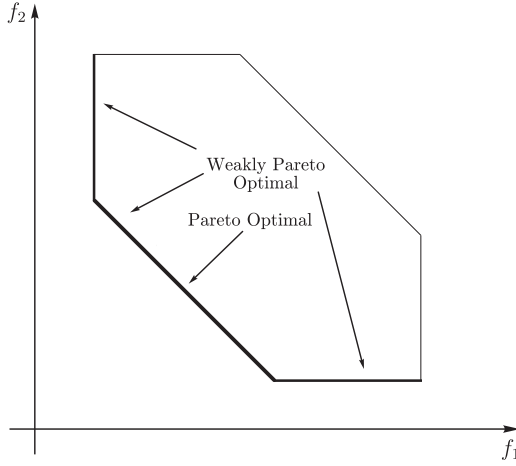


Figure 10. PS and weakly PS. Note that the PS is a subset of the weakly PS.

5.2. Definitions

The problem that arises in MOPs is that direct comparison of two objective vectors is not as straightforward as in single-objective problems. In single-objective problems when both $\mathbf{x}, \tilde{\mathbf{x}} \in S$ and $f(\tilde{\mathbf{x}}) < f(\mathbf{x})$, it is clear⁵ that the decision vector $\tilde{\mathbf{x}}$ is superior to $\mathbf{x}$. This is not the case when two or more objectives are considered simultaneously and there exists no *a priori* preference toward a particular objective.

If the relative importance of the objectives is unspecified, one way to partially order the objective vectors, $\mathbf{z} \in Z$, is to use the Pareto⁶ dominance relations, initially introduced by Edgeworth (1881) and further studied by Pareto (1896). Specifically, in a minimisation context, a decision vector $\tilde{\mathbf{x}} \in S$ is said to be **Pareto optimal** if there is no other decision vector $\mathbf{x} \in S$ such that $f_i(\mathbf{x}) \leq f_i(\tilde{\mathbf{x}})$, for all i , and $f_i(\mathbf{x}) < f_i(\tilde{\mathbf{x}})$ for at least one $i = 1, \dots, k$. Namely there exists no other decision vector that maps to a clearly superior objective vector. Similarly, a decision vector $\tilde{\mathbf{x}} \in S$ is said to be **weakly Pareto optimal** if there is no other decision vector $\mathbf{x} \in S$ such that $f_i(\mathbf{x}) < f_i(\tilde{\mathbf{x}})$ for all $i = 1, \dots, k$ (see Figure 10). Furthermore, a decision vector $\tilde{\mathbf{x}} \in S$ is said to **Pareto-dominate** a decision vector $\mathbf{x}$ iff $f_i(\tilde{\mathbf{x}}) \leq f_i(\mathbf{x})$, $\forall i \in \{1, 2, \dots, k\}$, and $f_i(\tilde{\mathbf{x}}) < f_i(\mathbf{x})$, for at least one $i \in \{1, 2, \dots, k\}$, then $\tilde{\mathbf{x}} \preceq \mathbf{x}$. So, in terms of generalised inequalities, if $\mathbf{F}(\tilde{\mathbf{x}}) \preceq \mathbf{F}(\mathbf{x})$ and $\mathbf{F}(\tilde{\mathbf{x}}) \neq \mathbf{F}(\mathbf{x})$, then $\tilde{\mathbf{x}} \preceq \mathbf{x}$. Also, a decision vector $\tilde{\mathbf{x}} \in S$ is said to **strictly dominate**, in the Pareto sense, a decision vector $\mathbf{x}$ iff $f_i(\tilde{\mathbf{x}}) < f_i(\mathbf{x})$, $\forall i \in \{1, 2, \dots, k\}$, then $\tilde{\mathbf{x}} < \mathbf{x}$. That is, if $\mathbf{F}(\tilde{\mathbf{x}}) < \mathbf{F}(\mathbf{x})$, then $\tilde{\mathbf{x}} < \mathbf{x}$. It should be noted at this point that when $<, \preceq$ are used in decision space, their meaning is mostly symbolic and is used to reflect the dominance relations in the objective space. The Pareto dominance relations with respect to a solution in objective space, $\mathbf{z}_p$, are depicted in Figure 11.

Most MOP solvers attempt to identify a set of Pareto optimal solutions; this set is a subset of the *Pareto optimal*

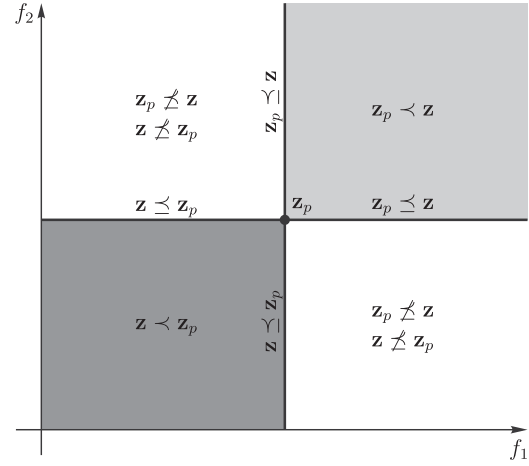


Figure 11. Pareto dominance relations in the objective space. The dark-grey region contains clearly better solutions compared with $\mathbf{z}_p$, the light-grey region clearly worse solutions, while the two other regions contain solutions incomparable to $\mathbf{z}_p$.

set (PS) which is also referred to as PF. The PS is defined as follows: $\mathcal{P} = \{\mathbf{z} : \nexists \tilde{\mathbf{z}} \preceq \mathbf{z}, \forall \tilde{\mathbf{z}} \in Z\}$. The decision vectors that correspond to the set, $\mathcal{P}$, are also called the Pareto set and are denoted by $\mathcal{D}$, namely $\mathbf{F} : \mathcal{D} \rightarrow \mathcal{P}$. The most widely used measures of the *quality* of the obtained PF are the generational distance, its inverted version (Van Veldhuizen 1999) and the hypervolume indicator (Zitzler and Thiele 1999). A detailed discussion of these and other metrics is beyond the scope of this work; however, there are a number of excellent works that address this issue, for example, Zitzler, Thiele, Laumanns, Fonseca, and da Fonseca (2003) and Zhou et al. (2011).

6. Methods for extending PBOTs to MOPs

In what follows, the main methodologies used to extend PBOTs to address MOPs are discussed. These are categorised into three broad categories, namely

- Pareto-based methods

Pareto-based methods employ Pareto-dominance relations to evaluate the quality of the population. These methods are still used to this day; however, it would appear that their ability to handle problems with more than three objectives (many-objective problems) is somewhat limited (Ishibuchi, Tsukamoto, and Nojima 2008).

- Decomposition-based methods

Decomposition methods employ a scalarising function and a set of weighting vectors to *decompose* an MOP into a set of single-objective subproblems. Upon solution of this set of subproblems, it is hoped that a good approximation of the PF is obtained. There is evidence to suggest that this way of dealing

with MOPs is much more scalable for many-objective problems. However, there are still difficulties to be resolved for this type of methods. For example, the distribution of solutions on the PF is controlled by the selection of weighting vectors. This issue is currently being addressed (Jiang, Cai, Zhang, and Ong 2011a; Jiang, Zhang, and Ong 2011b; Giagkiozis, Purshouse, and Fleming 2012; Gu, Liu, and Tan 2012; Giagkiozis, Purshouse, and Fleming 2013), but there are still many unresolved questions.

- Indicator-based methods

This type of methods for MOPs is also promising, for example, see Wang, Purshouse, and Fleming (2012), and are based on metrics developed to measure the *quality* of the solution set obtained from a PBOT. The most prevalent of these indicators has been the hypervolume indicator which was introduced in the context of multi-objective optimisation by Zitzler and Thiele (1999).

Schaffer (1985) was the first to extend GAs to become MOP solvers. In retrospect, his approach may appear simple, but for its time it was quite a conceptual leap since VEGA considered all objectives simultaneously. VEGA, see Algorithm 4, partitioned the population $\mathbf{x}$ into k equally sized randomly selected subpopulations, where k is the number of objectives. Then each partition is assigned to an objective and individuals are ranked according to their performance for the corresponding objective and a mating pool is formed using proportionate selection as described in Goldberg (1989). Crossover and mutation operate on the

Algorithm 4 VEGA

```

1:  $\mathbf{X}^{(1)} \leftarrow \text{Initialise}$ 
2:  $G \leftarrow 1$ 
3:  $ps \leftarrow N/k$  (Partition size)
4: repeat
5:    $\mathbf{X}^{(G)} \leftarrow \text{Shuffle}_{rows}(\mathbf{X}^{(G)})$ 
6:   for  $i \leftarrow 1, k$  do (Partition the population)
7:      $D_i \leftarrow \mathbf{X}_{ps \times (i-1)+1 \rightarrow i \times ps}^{(G)}$ 
8:      $E_i \leftarrow \text{Evaluate}(D_i)$ 
9:      $S_i \leftarrow \text{Proportionate selection}(E_i, D_i)$ 
10:  end for
11:   $\tilde{\mathbf{X}}^{(G)} \leftarrow \text{Join}(S_1, \dots, S_k)$ 
12:   $\tilde{\mathbf{X}}^{(G)} \leftarrow \text{Crossover}(\tilde{\mathbf{X}}^{(G)})$ 
13:   $\mathbf{X}^{(G+1)} \leftarrow \text{Mutate}(\tilde{\mathbf{X}}^{(G)})$ 
14:   $G \leftarrow G + 1$ 
15: until  $G \leq \text{MaxGenerations}$ 

```

entire population in the hope that linear combinations of the fitness functions would arise, thus estimating the entire PF. This, however, was to some extent problematic since a phenomenon called *speciation* (Schaffer 1985) did on some occasions occur. For instance, in objective functions with

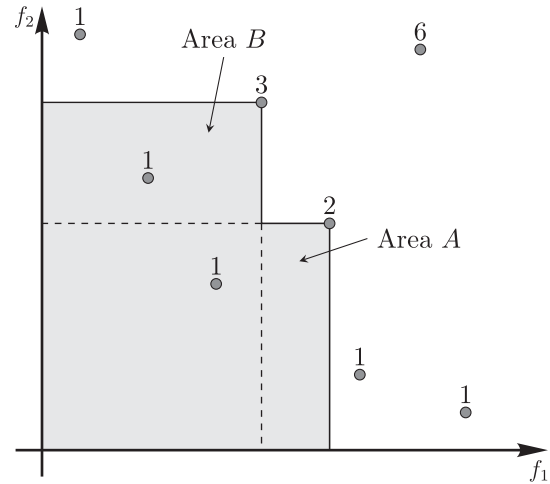


Figure 12. Ranking method used in MOGA. The numbers above the points represent the rank of the individual that results in that objective vector. The worst rank possible is N .

a concave PF, VEGA fails to approximate the PF as parts of the population drift toward the edges favouring one of the objective functions. Other methods employing the same or very similar approach are due to Fourman (1985) and Kursawe (1991), a variant applied to evolution strategies. A more recent work investigating VEGA is due to Toroslu and Arslanoglu (2007).

6.1. Pareto-based methods

Fonseca and Fleming (1993) were the first to use Pareto dominance relations in a multiple-objective genetic algorithm (MOGA), while incorporating progressive preference articulation enabling the DM to guide the search interactively. The dominance relation was used to rank the individuals in a population in a way similar to a set of selection methods proposed by Fourman (1985). Every individual in the population, after evaluation of the objective function, is ranked using the following relation:

$$r_i = 1 + p_i, \quad (19)$$

where p_i is the number of individuals dominating the decision vector $\mathbf{x}_i$. The idea behind this method of ranking, see Figure 12, is that misrepresented sections of the PF will increase the selection pressure to that direction of the front. As an example, the objective vector with rank 3, as seen in Figure 12, is dominated by two individuals, while the objective vector with rank 2 is dominated only by one. Alternatively, this means that within area A there is only one solution, while in area B there are two, suggesting higher concentration of solutions. This information is used to induce better spread in the objective vectors that are not part of the current PF approximation so as to maintain a relatively even *supply* of objective vectors in all regions of

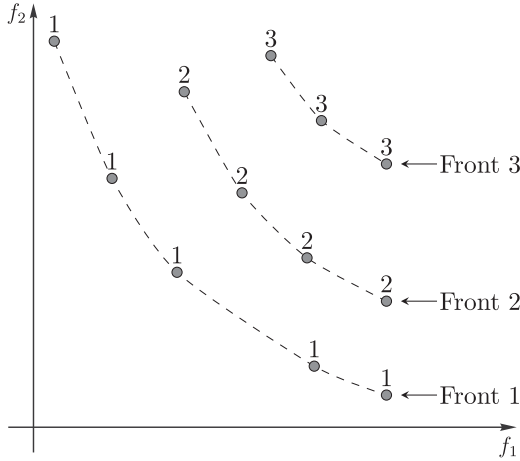


Figure 13. Non-dominated sorting method as used in NSGA.

the PF. For the objective vectors that are part of the PF approximation, Fonseca and Fleming introduced an adaptive factor, σ_{share} to penalise objective vectors that are less than σ_{share} apart. Finally, a facility for progressive preference articulation was embedded in MOGA providing the means for the DM to narrow down the search to *interesting* regions of the PF (Fonseca and Fleming 1993, 1995). This facility is implemented by incorporating a goal attainment method in the ranking procedure. Let $\mathbf{g} = \{g_1, \dots, g_k\}$ be the goal vector and $\mathbf{z}_1 = \{z_1, \dots, z_k\}$ and $\mathbf{z}_2$ be two objective vectors. A *preference* relation can be expressed such that when $k - s$ of the k objectives are met, then $\mathbf{z}_1$ is *preferable* to $\mathbf{z}_2$ if and only if

$$\begin{aligned} & z_{(1,1\dots k-s)} \leq z_{(2,1\dots k-s)} \text{ or} \\ & \{(z_{(1,1\dots k-s)} = z_{(2,1\dots k-s)}) \wedge \\ & [(z_{(1,k-s+1\dots k)} \leq z_{(2,k-s+1\dots k)}) \vee \\ & (z_{(2,k-s+1\dots k)} \not\leq g_{(k-s+1\dots k)})]\} \end{aligned} \quad (20)$$

The rest of the MOGA utilised Gray encoding for the chromosomes, two point reduced surrogate crossover and the standard binary mutation.

Another prominent algorithm, the non-dominated sorting GA (NSGA), that utilised the Pareto dominance relations was proposed by Srinivas and Deb (1994). This method is almost identical to the non-dominated sorting idea proposed by Goldberg (1989). Non-dominated sorting is very similar to a well-established concept in non-cooperative game theory in which candidates need to select a *winning* strategy while considering what their opponents' strategy might be. This idea is usually referred to as *eliminating dominated strategies* which is an iterative process of deleting dominated strategies that, if chosen, would lead to lower payoffs relative to the opponents and thus result in unfavourable outcomes. Interested readers are referred to Bernheim (1984). Non-dominated sorting, see Figure 13, works as follows:

Step 1 Evaluate the objective function for each individual in the population $\mathbf{X}$.

Step 2 Find the non-dominated individuals in the population, $\mathbf{X}_{\text{nd}}$, and remove them from the current population $\mathbf{X}_{\text{new}} = \{\mathbf{x} : \mathbf{x} \in (\mathbf{X} \cap \mathbf{X}_{\text{nd}})^c\}$.

Step 3 Repeat **Step 2** until no solutions remain in $\mathbf{X}_{\text{new}}$.

Each non-dominated set identified by this process is labelled as a *front* and given a rank according to the position it has in the iteration. For example, the first identified *front* would be *front 1*, the second *front 2*, and so on. Subsequently, the individuals of each front are assigned fitness values using the following procedure starting from the first *front*:

Step 1: Assign to all the individuals in *front 1* a fitness value N , where N is the number of individuals in the population.

Step 2: Use sharing to penalise clustered solutions in the current *front*.

Step 3: Assign the lowest fitness minus a small value ϵ to all individuals in the next *front* and go to **Step 2** until all individuals in all fronts have been assigned a fitness value.

The sharing distance σ_{share} is calculated based on the distance in the decision variable space rather than the objective space and is held fixed throughout the algorithm execution. Finally, NSGA employed the roulette-wheel selection operator (Goldberg 1989) and the usual bitwise mutation operator (see Figure 2).

The fact that NSGA uses sharing based on the distance of the decision vectors seems counterintuitive since equally spaced decision vectors do not necessarily map to equally spaced objective vectors, unless the mapping is affine which is usually not the case for most objective functions. This can potentially mislead the algorithm as to which solutions are densely clustered and which are not, resulting in exactly the opposite effect. Although, as the authors state, sharing can be performed based on the objective vectors (Srinivas and Deb 1994); they do not mention how σ_{share} would be selected in that case. Non-dominated sorting is the basis of an improved version of NSGA, namely NSGA-II (Deb, Pratap, Agarwal, and Meyarivan 2002), which is actively used for benchmarking and improved upon to this day (see Zhang and Li 2007; Li and Zhang 2009; Ben Said, Bechikh, and Ghédira 2010; Bui, Abbass, Barlow, and Bender 2012; Wang et al. 2012).

One problem identified for Pareto-based methods for MOPs was that good solutions could be lost if they were not retained using some mechanism and the cost to rediscover them can potentially be prohibitive. This situation is exacerbated when the cost, computational or otherwise, of evaluating the objective function is high; this is often the case in most real-world applications. A solution to this

predicament is the use of *elitism*, in other words, the retention of highly performing individuals in the population. Zitzler and Thiele (1999) introduced an algorithm, named the strength Pareto evolutionary algorithm (SPEA), which used elitism. Their approach was not the first to utilise an external archive to retain good performing individuals; however, it was certainly one of the earliest and most elegant attempts.

SPEA, in addition to the current population $\mathbf{X}$, maintains an archive population $\tilde{\mathbf{X}}$ of maximum size $\tilde{N}$ which is usually smaller than the population size N . Zitzler and Thiele (1999) suggest $\tilde{N} = 0.25N$. At the start of the algorithm, the external archive is empty and the population is initialised randomly within the decision variable limits or as appropriate to the problem. After the objective function has been evaluated, the population is searched for non-dominated solutions which are copied to the archive $\tilde{\mathbf{X}} = \mathbf{X}_{\text{nd}}$. This direct assignment is performed only on the first iteration of the algorithm. In subsequent iterations, the archive updating procedure is different if the total population in the archive exceeds $\tilde{N}$. When the archive size becomes larger than $\tilde{N}$, the superfluous solutions are removed based on a crowding algorithm that the authors of SPEA introduced. This crowding algorithm maintains the *elite* solutions that have maximal spread along the PF. After the archive size is properly reduced to its maximum size, the population and the *elite* individuals are assigned fitness values in the following way:

Step 1: Assign fitness values, called *strength* (s), to the population in the archive $\tilde{\mathbf{X}}$, using the following relation: $s_i = \frac{n_i}{N+1}$, where n_i is the number of individuals that the i th solution in the archive dominates in the regular population $\mathbf{X}$.

Step 2: The regular population $\mathbf{X}$ is then assigned fitness values according to $f_i = 1 + \sum_{\mathbf{x}_j \preceq \mathbf{x}_i} s_j$. Note that in SPEA a lower fitness value is considered to be better. Therefore, if a decision vector $\mathbf{x}_i$ is not dominated by any individual in the population $\mathbf{X}$, its fitness, f_i , would be equal to 1.

The way the archive is maintained and used in SPEA ensures that the population $\mathbf{X}$ is rewarded when it approaches the PF. In a way the archive is used as a *moving target* to which the regular population aspires. Again the rest of the algorithm utilises roulette wheel selection and random bit mutation.

6.2. Decomposition-based methods

As mentioned in Section 6, decomposition methods depend on scalarising functions to break down an MOP into a set of single-objective subproblems. The premise of this approach is that the methods described in Section 4 can be applied

almost unaltered. This benefit, however, takes its toll as the selection of the weighting vectors controls the distribution of solutions on the PF (Giagkiozis et al. 2012, 2013). This issue has been investigated in some detail in Giagkiozis et al. (2013), where it is shown that the algorithm scalability to many-objective problems can be significantly affected if an arbitrary method is employed in the selection of the weighting vectors.

A family of scalarising functions is the weighted metrics method (Miettinen and Mäkelä 2002):

$$\min_{\mathbf{x}} \left(\sum_{i=1}^k w_i |f_i(\mathbf{x}) - z_i^*|^p \right)^{\frac{1}{p}}, \quad (21)$$

where $\mathbf{w} = (w_1, \dots, w_k)$ is referred to as the weighting vector and w_i are the weighting coefficients. The weighting coefficients can be viewed as factors of relative importance of the scalar objective functions in $\mathbf{F}(\cdot)$. The weighting coefficients must be $w_i \geq 0$ and $\sum_{i=1}^k w_i = 1$, also $p \in [1, \infty)$. However, p is usually an integer or equal to ∞ . A potential drawback of weighted metrics based scalarising functions is that the ideal vector, $\mathbf{z}^*$, has to be known *a priori*. However, this vector can be estimated adaptively during the process of optimisation (Zhang and Li 2007). When $p = \infty$ the Chebyshev scalarising function is obtained as

$$\min_{\mathbf{x}} \|\mathbf{w} \circ |\mathbf{F}(\mathbf{x}) - \mathbf{z}^*|\|_{\infty}. \quad (22)$$

The $\circ$ operator denotes the Hadamard product which is element-wise multiplication of vectors or matrices of the same size. The key result that makes (22) very interesting is that for every Pareto optimal solution there exists a weighting vector with coefficients $w_i > 0$, for all $i = 1, \dots, k$ (Miettinen 1999), meaning that all Pareto optimal solutions can be obtained using (22). This result is quite promising, although in current practice the choice of weighting vectors is made primarily using *ad hoc* methods (see Das and Dennis 1996; Jaszkiewicz 2002; Zhang and Li 2007), and so the points on the PF cannot be controlled effectively.

The IS-based algorithm proposed by Yoo and Hajela (1999) used the weighting method (Miettinen 1999, p. 27), which is obtained by setting $p = 1$ and $\mathbf{z}^* = \mathbf{0}$ in (21). The weighting method aggregates all the objective functions into a single function using the following relation:

$$\begin{aligned} \min_{\mathbf{x}} g(\mathbf{x}) &= \sum_{i=1}^k w_i f_i(\mathbf{x}) \\ \sum_{i=1}^k w_i &= 1, \text{ and, } w_i \geq 0, \text{ for all } i \\ &\text{subject to } \mathbf{x} \in S, \end{aligned} \quad (23)$$

where w_i represents the weight of the i th objective function. When all of the objective functions are equal in importance to the DM, the weights can be set to $w_i = \frac{1}{k}$, where k is the number of objective functions. Using the weighting method results in an approximate solution for one point of the PF. To explore more points, various combinations of the weights have to be used. Yoo and Hajela (1999) randomly generated a set of weight combinations in an attempt to uniformly sample the PF, and then used these different weighting vectors to guide the search.

Some difficulties encountered in the above implementation are that the weighting method is used and its weaknesses could affect the algorithm in several ways. For instance, evenly spaced weighting vectors do not necessarily produce an even distribution of points along the PF and two widely differing weight vectors need not produce significantly different points along the PF (Miettinen 1999). An additional shortcoming of the weighting method is that it cannot guarantee that all the Pareto optimal points can be obtained (Miettinen 1999, p. 80). It is apparent even in this early application of AIs to MOPs that there is a *jump* in algorithm complexity when compared to a GA. This complexity inevitably leads to an increased demand on computational resources. This cost will have to be justified especially since GAs, as can be seen in Fonseca and Fleming (1993), can explicitly deal with constraints as well. However, the computational cost of most algorithms can often be ignored when compared with the cost of evaluating the objective function. Nevertheless, this increase in algorithm complexity is not due to the use of the weighting method.

Parsopoulos and Vrahatis (2002) introduced a generalisation of PSO to MOPs based on a methodology presented by Jin, Olhofer, and Sendhoff (2001) using some variations of the weighting method (see Equation (23)). Jin et al. (2001) argued that weighting methods, or more specifically the conventional weighting method, could not approximate a concave PF due to the fact that the surface of a concave PF is *unstable* and attracts solutions toward the two extremes⁷ for which the weights are (0, 1) and (1, 0), respectively; this view is further supported in Giagkiozis and Fleming (2012). However, in real-world applications, the PF *shape* is usually not known *a priori*. To deal with this predicament, Jin et al. (2001) suggested that if the optimiser is run with weights of one of the two extremities of the PF, that is, either (0, 1) or (1, 0), and then gradually or abruptly the weighting vectors are exchanged, the optimiser should be able to traverse the entire PF. To capture the Pareto optimal solutions, Jin et al. suggested an archiving technique that would, hopefully, result in a good approximation of the entire PF. The problem with this technique is that the optimiser is effectively segmented in two phases, the first one with the fixed weighting vectors and the second one where the population is allowed to *slide* along the PF. To successfully accomplish this task, a metric is needed to measure the

convergence rate of the first phase so that the second phase is initiated. This task is not trivial because it presupposes that the minimum is already known. Another potential problem is that if disjoint regions in the decision variable space map to neighbouring objective values, this approach will have difficulty approximating the PF. Finally, it is difficult to envisage how would this method scale to problems with more than two objectives. Despite these difficulties, the aforementioned approach seems to perform reasonably well for the test problems used in Parsopoulos and Vrahatis (2002) and Jin et al. (2001).

As mentioned in Section 3, the main algorithm in PBOs is usually built with single-objective optimisation in mind and as such the extension to multiple objectives, as it will be apparent by now, requires a number of considerations. For instance, diversity-preserving operations and *eliteness*-preserving strategies inevitably result in higher computational costs, and for algorithms utilising directly the non-dominated sorting strategy (see Section 6.1), the cost is even higher. Relatively recently an MOEA based on decomposition (MOEA/D) was introduced by Zhang and Li (2007) as an alternative way of extending DE and EAs, in general, to deal with MOPs. The approach depends on one of several available decomposition techniques – weighted sum, Chebyshev (Miettinen 1999) and normal boundary intersection (Das and Dennis 1996) decompositions – with each having its own strengths and weaknesses. The minimisation problem from Section 5.1, when using the Chebyshev decomposition, can be restated as follows:

$$\begin{aligned} \min_{\mathbf{x}} g_{\infty}(\mathbf{x}, \mathbf{w}^s, \mathbf{z}^*) &= \max_{i=1 \dots k} (w_i^s |f_i(\mathbf{x}) - z_i^*|) \\ \text{for all } s &= \{1, \dots, N\}, \\ \text{subject to } \mathbf{x} &\in S, \end{aligned} \quad (24)$$

where $\mathbf{w}^s$ are N evenly distributed weighting vectors. The idea behind this is that g_{∞} is a continuous function of $\mathbf{w}$ (Zhang and Li 2007). The authors' hypothesis was that for N evenly distributed weighting vectors the same number of single-objective subproblems is generated and their simultaneous solution should result in evenly distributed Pareto optimal points, assuming the objectives are normalised (Zhang and Li 2007). MOEA/D has been very successful, and an updated version was the winner of the CEC'09 competition for unconstrained problems (Zhang, Liu, and Li 2009).

6.3. Indicator-based methods

The need for comparative analyses regarding the strengths and weaknesses of different algorithms led to the introduction of several *indicators*, see Zitzler et al. (2003), measuring various *qualities* of the resulting Pareto optimal set approximation. With the advent of such indicators, some researchers grasped the opportunity to utilise these within

the evolution process (see Zitzler and Künzli 2004; Wagner, Beume, and Naujoks 2007). One important indicator is a derivative of the Lebesgue measure (Halmos 1974), initially introduced in this context by Zitzler and Thiele (1999); this indicator is now commonly referred to as the hypervolume indicator (Bader and Zitzler 2008). The most favourable quality of the hypervolume indicator is its monotonicity with regard to Pareto dominance (Bader and Zitzler 2008). This implies that the hypervolume indicator can be used in place of Pareto dominance relations for fitness assignment. However, this indicator is not without drawbacks since most known algorithms to calculate the Lebesgue measure are of exponential complexity with respect to the number of objectives (Bader and Zitzler 2008): a limitation that imposes a restriction on the number of objectives that can be considered in an algorithm based on this indicator.

Recently Bader and Zitzler (2008, 2011) introduced an algorithm based on the hypervolume indicator for many-objective optimisation.⁸ Their idea is that precise calculation of the hypervolume is not required, *per se*, to enable a useful ranking of the population. Under this premise they developed a methodology which uses Monte Carlo sampling to approximately determine the value of the hypervolume and subsequently use this to assign fitness to the decision vectors. The results produced were quite favourable for HypE and the authors demonstrated that their proposed technique can effectively tackle as many as 50 objectives (Bader and Zitzler 2008).

6.4. Commentary on the methods

Pareto dominance based methods have been heavily criticised for their apparent inability to scale *gracefully* for an increasing number of objectives (Teytaud 2006; Ishibuchi et al. 2008; Hadka and Reed 2012). The main arguments are that the number of non-dominated solutions increases to a degree that any comparison becomes meaningless and there is no clear way for the search to continue (Ishibuchi et al. 2008). This view is further supported by theoretical arguments (Teytaud 2006; Giagkiozis and Fleming 2012). This, of course, is only true when all the scalar objectives are competing; otherwise, increasing the number of objectives may even render the problem easier to solve (Scharnow, Tinnefeld, and Wegener 2004; Brockhoff et al. 2007). Nevertheless, this issue is not exclusive to Pareto-based methods, rather it is a problem that all methods encounter, albeit to varying degrees (Giagkiozis and Fleming 2012). Another issue with Pareto-based methods is that of controlling the distribution of solutions on the PF. This is one of the reasons for the introduction of modifications to the Pareto-dominance definition such as ε -dominance (Laumanns, Thiele, Deb, and Zitzler 2002), cone ε -dominance (Batista, Campelo, Guimarães, and Ramírez 2011) and other methods (Zitzler, Laumanns, and Thiele 2001; Purshouse and Fleming 2007; Adra and Fleming 2009). All

these additions to the main algorithm can become quite costly as the number of objectives increases. An alternative is the use of decomposition-based methods, which transform this into a different problem – that of selecting the weighting vectors to produce a desired distribution (Jiang et al. 2011b; Giagkiozis et al. 2012, 2013; Gu et al. 2012; Tan, Jiao, Li, and Wang 2012;). In contrast, the benefit of indicator-based methods is that if an indicator that reflects well the preferences of the analyst and the DM is identified, then no additional considerations are necessary as the algorithm is using this metric directly. This, however, incurs a high computational cost (Zitzler et al. 2003; Bader and Zitzler 2008; Bader 2010) and it is unclear how the preferences of the DM can be incorporated.

7. Software libraries for population-based optimisation

There are a number of available software libraries for most of the algorithm families discussed in this work; however, as can be seen from Table 1, some algorithms are far better supported than others. The most striking observation in Table 1 is that it appears that no library, at least in the set of libraries we include here, has support for artificial ISs or ACO algorithms. This does not mean that there is absolutely no open source software available for these methods; it simply means that popular libraries have not included them explicitly. In our view, the reason for this is that ACO is closely linked with the problem representation which creates difficulties when creating an abstraction in order for it to be included in a library. Nevertheless, there are several implementations available (see Dorigo, Stützle, Farooq, and Blum 2013). No freely available implementation of AIS could be located; this is consistent with the observation that there are relatively few publications on this algorithm family (see Figure 14).

Although every library listed in Table 1 has its merits, jMetal (Durillo and Nebro 2011), PISA (Bleuler et al. 2003) and ParadisEO (Liefoghe et al. 2007) seem to be the most widely used by the community.

8. Discussion

There is little doubt that evolutionary algorithms have progressed with great pace over the last 30 years (see Figure 14). However, there is little advice available to the practitioner faced with the challenge of choosing a suitable algorithm. In a way this choice is an MOP in itself, and quite a challenging one. Nevertheless, the fact that there is an absence of concrete guidelines is not accidental and can be attributed to several factors. For instance, not all the methodologies mentioned are easy to study analytically and thus a rigorous and conclusive comparison is rather difficult. Also most studies are performed on test problems

Table 1. Software libraries for population-based optimisation methods.

Name	Language	License	GA	ES	AIS	ACO	DE	PSO	EDA	Reference
ECJ	Java	AFLv3	✓	✓	×	×	✓	✓	×	Panait et al. (2010)
Opt4J	Java	LGPL	✓	×	×	×	✓	✓	×	Lukasiewicz, Glaß, Reimann, and Teich (2011)
JGAP	Java	GPL	✓	×	×	×	×	×	×	Meffert and Rotstan (2010)
EvA2	Java	LGPL	✓	✓	×	×	✓	✓	×	Kronfeld, Planatscher, and Zell (2010)
jMetal	Java	LGPL	✓	✓	×	×	✓	✓	×	Durillo and Nebro (2011)
EO	C++	LGPL	✓	✓	×	×	✓	✓	✓	Keijzer, Merelo, Romero, and Schoenauer (2002)
MOMH Lib++	C++	LGPL	✓	×	×	×	×	×	×	Jaszkiewicz and Dabrowski (2012)
Open Beagle	C++	LGPL	✓	✓	×	×	×	×	×	Gagné and Parizeau (2006)
ParadisEO	C++	CeCILL	✓	✓	×	×	✓	✓	✓	Liefooghe, Basseur, Jourdan, and Talbi (2007)
Shark	C++	GPL	×	✓	×	×	×	×	×	Igel, Heidrich-Meisner, and Glasmachers (2008)
PISA	C	Mixed	✓	×	×	×	×	×	×	Bleuler, Laumanns, Thiele, and Zitzler (2003)
Heuristic Lab	C#	GPL	✓	✓	×	×	×	✓	×	Wagner et al. (2010)

such as the ZDT problems (Zitzler, Deb, and Thiele 2000), the DTLZ problems (Deb et al. 2002) or the WFG problem set (Huband et al. 2006) and others (Li and Zhang 2009). Furthermore it is rare to find a real-world application in the literature that encompasses the true complexity of problems encountered in practice (Michalewicz 2012). There is no dispute over the point that there are reported applications in the literature (see e.g. Zhou et al. 2011). However, the scale of such problems pales in comparison to the size

of problems that interior point methods can address, to the extent that Michalewicz (2012) calls them *toy problems*. To give a perspective of the size of this gap, consider that interior point methods can solve problems with several million decision variables, constraints and objectives, while their convergence rate remains almost unaffected (Gondzio 2012). To this day evolutionary algorithms mostly deal with problems with two or three objectives with relatively few addressing more, but no more than approximately 10, and

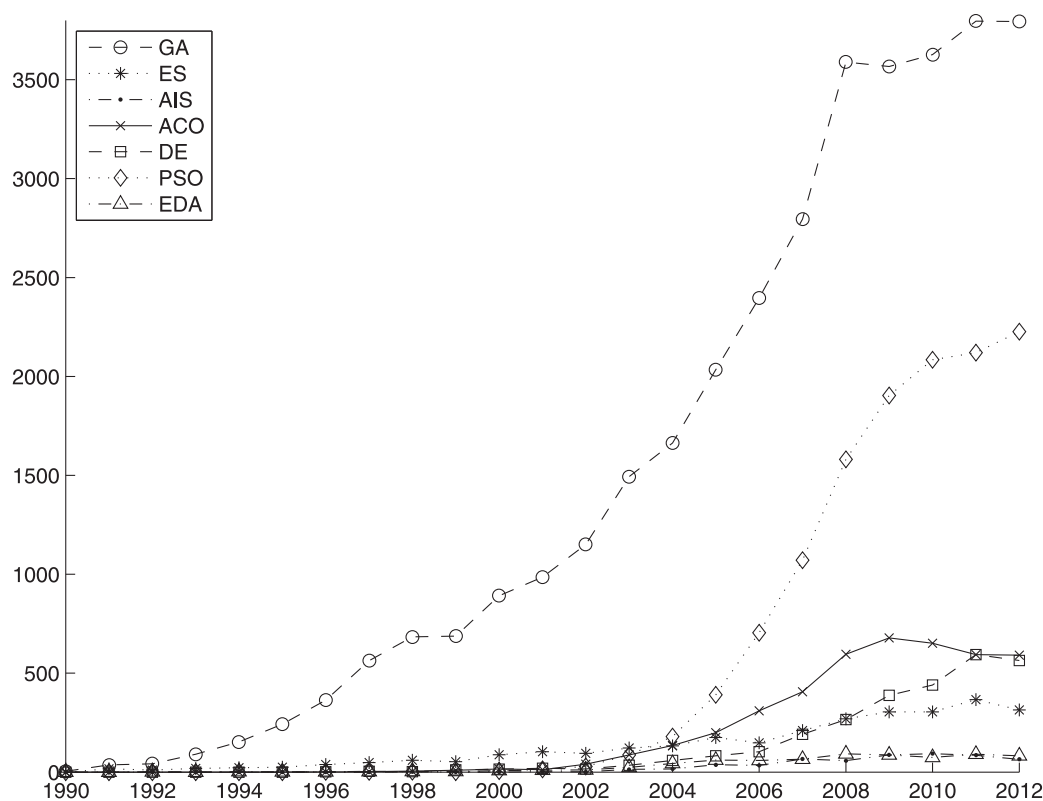


Figure 14. Number of publications per year for each algorithm family. Results obtained from ISI Web of Knowledge. The queries used to obtain the number of papers can be found in Appendix A.

with fewer than 100 decision variables (Chiong and Kirley 2012; de Lange, Samoilovich, and van der Rhee 2012; Kim, Han, Kim, Choi, and Kim 2012; Tan et al. 2012; Wei, Oon, Zhu, and Lim 2012). Moreover, there is not a single study available⁹ that establishes a strong positive correlation between superior algorithm performance on test problems and real-world (and real-scale) problems. However, it would be an unfair statement to say that the community is unaware of these issues. A strong indication of this awareness, and a positive outcome in our view, is the development of evolutionary algorithms based on surrogate models (Poloni, Giurgevich, Onesti, and Pediroda 2000; Jin, Olhofer, and Sendhoff 2002; Ong, Nair, and Lum 2006; Adra, Dodd, Griffin, and Fleming 2009; Lim, Jin, Ong, and Sendhoff 2010). The main difference of such algorithms is that they mostly operate using a model of the objective function, which is potentially very expensive to evaluate. This approach is taken a step further by using a convex surrogate model (Wanner, Guimarães, Takahashi, and Fleming 2008; da Cruz, Cardoso, Wanner, and Takahashi 2011). The benefits of such an approach should be obvious, in light of our discussion on interior point methods. Some may argue that by using such a simple objective function, a quadratic in the case of Wanner et al. (2008), much is lost in terms of fidelity of the solution. However, if such a method is combined with the scalability of interior point methods, the fact that a simple model is used is overshadowed by the potentially much larger scalability of the resulting algorithm which would allow the practitioner to consider the problem in its entirety and not just a small part of it. This in turn may help lessen the gap between theory and reality that Michalewicz (2012) is talking about. Nevertheless, not much work has been done toward the direction proposed by Wanner et al. (2008) so its merits are yet to be evaluated in practice. The algorithms we mention in this paragraph are on some occasions referred to as memetic algorithms, and their main characteristic is that they incorporate some form of local search – often gradient based – to improve solutions. For an excellent survey of this topic, the reader is referred to Chen, Ong, Lim, and Tan (2011).

In this section, as an initial guide to the practitioner, we summarise, and to some extent justify, the advantages and disadvantages of the algorithms: see Table 2. Although every possible effort has been made to validate the features and their corresponding ratings in Table 2, the ratings presented inevitably have some bias introduced by the authors. This fact is ascribed to the sheer volume of material available¹⁰ relating to population-based multi-objective optimisation algorithms, rendering an attempt to thoroughly compare the different algorithms extremely difficult, if not impossible.

The basis for the assigned ratings in Table 2 is now addressed. Factors that influenced these ratings are the following:

- Degree of support for a feature

The degree of support for a feature is determined by how well established it is for algorithms in a particular family. By ‘established’, we mean that the research community consistently improves upon the feature and that this process, relative to the study interval (past 30 years), is not entirely novel. For example, AISs have a score of 2 for continuous problems since this extension is very recent (Castro and Von Zuben 2010a,b) and involves hybridisation of the underlying algorithmic framework. For the same reason, all algorithmic families score poorly for mixed-type decision variables (Table 2: Mixed), since systematic studies on this topic are extremely scarce.

- Reported performance

Another influencing factor is the reported performance, within the body of references in this work, of individual methods. For example, there is a clear way to *bias* the model-building process in EDAs so as to encompass prior information when compared with any of the other algorithm families; hence, they are assigned a top score in Table 2: Prior information. To some extent, this argument is also true for ACO. Also, for combinatorial, continuous, discrete and mixed-type problems, we have used the queries in Appendix A to find the proportion of published work that addresses these problem types for each algorithm family (Figure 15); this was also taken into account as it is an indicator of the research focus of the community. Our assumption is that, if the proportion of the research output in a family of algorithms is higher for a particular class of problems, then there is a better chance for practitioners to identify a work that addresses a similar problem and that the potential for new positive developments is also increased.

Problem types

Optimisation problems can be broadly placed in four categories:

- Continuous – problems with continuous decision variables.
- Discrete – problems with discrete decision variables.
- Mixed – problems with one or more discrete decision variables with the remaining variables being continuous.
- Combinatorial

The algorithm families reviewed in this work support these categories in varying degrees. In what follows, we provide a commentary on their support for these problem types.

Table 2. Relative strengths and weaknesses of algorithm families for MOPs. The values in the table are relative to other algorithm families compared with the support for the particular feature. A value of 5 translates that this particular family of algorithms is very well suited for this type of problem or in general that support for a feature is strong. A ‘-’ signifies that there is no support for a particular feature.

Features	Algorithm Family						
	GA	ES	AIS	ACO	DE	PSO	EDA
Continuous	5	5	2	1	5	5	4
Discrete	5	1	5	5	2	2	5
Mixed	1	1	—	1	1	1	1
Combinatorial	2	3	3	5	2	2	4
Complexity	4	4	3	2	5	4	1
Cost	4	4	3	3	5	5	1
P. Sensitivity	2	3	2	3	4	3	4
No of Parameters	2	3	2	3	4	4	5
MOS	5	4	4	2	5	3	2
Prior Information	2	2	3	4	2	2	5

Continuous

ES has supported this type since its inception (Rechenberg 1965), the same is true for PSO (Eberhart and Kennedy 1995) and DE (Storn and Price 1995), while GAs have been extended by means of several crossover and mutation operators for continuous problems (Deb and Agrawal 1994; Voigt et al. 1995; Ono and Kobayashi 1997). EDA is ranked at 4 since its support for real decision variables is

relatively recent (Bosman and Thierens 2000; Thierens and Bosman 2001). AIS-based algorithms have adopted continuous decision variables quite recently (Castro and Von Zuben 2010a,b). However, the employed methodology resembles EDAs since a Gaussian network model (Castro and Von Zuben 2010a) is created to enable the underlying AIS method deal with this type of decision variable. This means that the method in Castro and Von Zuben (2010a) is in

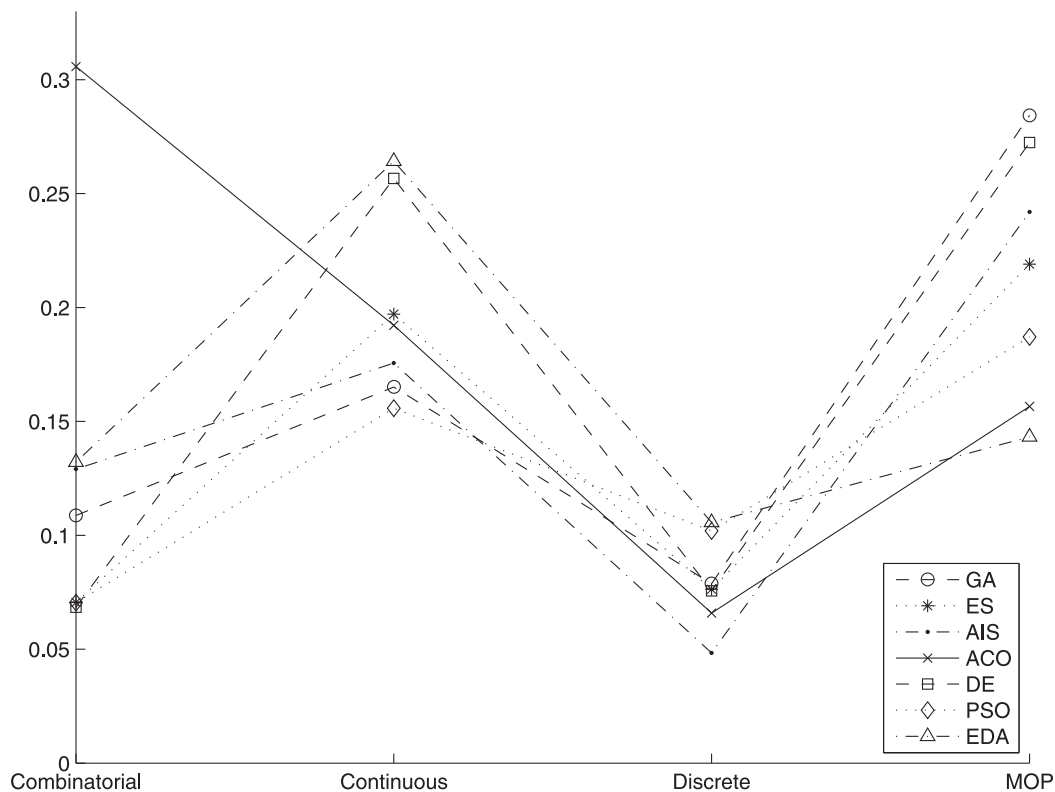


Figure 15. Proportion of papers released with mention to a specific feature in all the algorithm families. These results were extracted from ISI Web of Knowledge and the employed queries can be found in Appendix A.

essence a hybrid of two algorithmic families, namely AISs and EDAs. Admittedly, the reported performance in Castro and Von Zuben (2010b) is comparable with, and in some instances better than, the compared algorithms. For this reason, AISs score 2 in Table 2, since their ability to handle continuous decision variables is much better than ACO, but research involving AISs with continuous decision variables is quite sparse. ACO has relatively poor support for this type (Bilchev and Parmee 1995; Socha and Dorigo 2008).

Discrete

GAs initially used discrete representation (Holland 1975) as well as AISs (Farmer et al. 1986), EDAs (Baluja and Caruana 1995) and ACO (Colorni, Dorigo, and Maniezzo 1991). ESs, DE and PSO have been developed for problems with continuous decisions variables (Eberhart and Kennedy 1995; Storn and Price 1995); however, some extensions exist that extend these classes of algorithms to tackle problems requiring discrete decision variables, see Pan, Tasgetiren, and Liang (2007) for DE, Cai and Thierauf (1996) for ES and Kennedy and Eberhart (1997) for PSO.

Mixed

Tracking published resources in this category proved challenging. Although, in the authors' experience, mixed-type decision variables are often necessary for real-world applications, the literature is very scarce on this topic. This fact is reflected in the relative rankings, which, for all algorithmic families, are the same. One extension of ACO to mixed-type decision variables is presented in Socha (2004), and an application using EDAs in Ocenasek and Schwarz (2002). Regarding ESs there are some studies dealing with mixed-integer decision variables (Bäck, Schütz, Ack, and Utz 1995; Emmerich, Grötzner, Groß, and Schütz 2000); however, the bulk of the research is directed toward real decision variable problems. Additionally, some examples in DE and PSO are Pampara et al. (2006), Lampinen and Zelinka (1999) and Gaing (2005). GAs have the potential to excel here as the abstraction used to represent decision variables (chromosome) can be intuitively extended.

Combinatorial

To some extent all the studied algorithmic families support or have the ability to tackle combinatorial problems, although ACO has a strong lead in this type of problems (Bell and McMullen 2004). Also ACO papers consistently feature in the top 10 most cited papers on combinatorial optimisation problems (SCOPUS 2012). EDAs are ranked 4 only because there seems to be a smaller body of research compared with ACO in this category. However, EDAs are gaining ground quite rapidly (SCOPUS 2012). Some exam-

plars of successful applications can be found in Larranaga and Lozano (2002) and Hauschild and Pelikan (2011). Regarding the rest of the families and considering the fact that most are employed in combinatorial problems in a hybrid form with local search techniques (Hertz and Widmer 2003; Puchinger and Raidl 2005), ESs and AISs are given a slight advantage due to their increased use (SCOPUS 2012). However, this point is debatable since it has been shown (Zhang and Li 2007) that a framework based on decomposition, which can utilise GAs or DEs as its main search algorithm, does perform admirably on multi-objective knapsack problems.

Applicability

Complexity – in terms of implementation

By far the easiest techniques to implement are DEs (Storn and Price 1995) and PSO (Eberhart and Kennedy 1995) followed by ESs (Rechenberg 1965), GAs and AISs. ACO is given a complexity of 2 due to the fact that the practitioner usually has to adapt the algorithmic process significantly to solve a particular problem (López-Ibáñez and Stützle 2010) and EDAs are deemed to be the most complex to implement since quite elaborate techniques are required (Hauschild and Pelikan 2011). It should be noted that these ratings apply to the *main algorithm* and not to extensions to MOPs since the total complexity would be greatly affected by the selected methodology.

Cost – in terms of required resources

This more or less mirrors the implementation complexity which is fairly reasonable. Although it should be stated that EDAs have much lower memory requirements (Hauschild and Pelikan 2011), their computational cost per iteration is higher in comparison with others in the algorithm families. This is especially true when more elaborate probabilistic models are created, which is the case for hierarchical-BOA (Pelikan, Goldberg, and Tsutsui 2005) and RM-MEDA (Zhang, Zhou, and Jin 2008). DE and PSO have the highest score, since their updating rules do not require elaborate calculations as can be found in Sections 4.5 and 4.6. Following these are ESs and GAs, which are still relatively not very demanding. For example, ESs use normal distributions which are more expensive to calculate compared with the small number of addition and multiplication operations in DE and PSO, and GAs have some crossover operators that can slightly increase their cost, for example, Deb and Agrawal (1994).

Number of parameters requiring selection

For this feature, we stress the point that the main algorithms are assessed and not their many extensions to MOPs, with

or without constraints. Here DE (Storn and Price 1995), EDAs (Hauschild and Pelikan 2011) and ESs (Rechenberg 1965) score highest due to their inherent adaptive nature. EDAs are awarded the top score since most of their updating rules are calculated during the algorithm execution based on some measure of optimality, and such measures can be found in abundance in statistics. For example, optimal updating rules can be formulated using the Kullback–Leibler divergence as is the case for the cross-entropy method (De Boer, Kroese, Mannor, and Rubinstein 2005). Therefore, since there are ways to *optimally* update the *direction* of search, the need for controlling parameters is, relative to other families, less significant.

Multi-objective scalability

In this category, ACO is ranked 2 since it seems that extending ACO to more than two or three objectives is extremely difficult (López-Ibáñez and Stützle 2010) or at least no attempts have been brought forward so far.¹¹ This is also supported by the fact that articles for multi-objective optimisation algorithms based on ACO do not appear in the top 20 cited papers (SCOPUS 2012). For the rest, a rank of 4 was given if the methodologies to extend a particular family have been relatively recently addressed; such is the case for EDAs, PSO and AISs. It seems natural that GAs and ESs score highly here since their techniques have the longest history. Additionally, regarding EDAs, the problem is that the method used to extend this family to MO problems affects the complexity of the probabilistic model. For instance, if a Bayesian network is used as in Pelikan and Sastry (2006), it is not very easy to envisage how this will be directly extended to many objectives.

Use of prior information

The highest scores for this feature are assigned to ACO- and EDA-based algorithms. For EDAs this is due to the ease with which prior information can be incorporated (Hauschild and Pelikan 2011). This type of bias can be induced by altering the initial parameters of the probabilistic model (Hauschild and Pelikan 2011). ACO also has probabilistic transitional rules; hence to bias them towards promising regions is relatively straightforward and is common practice when the problem is formulated, for example, see López-Ibáñez and Stützle (2010). In the remainder of the reviewed algorithm families, prior information about a problem has to be embedded, usually on an ad hoc basis. For example, a GA/DE-based algorithm MOEA/D (Zhang and Li 2007) required significant changes in order to be applied to the multi-objective 0–1 knapsack problem.

9. Conclusion

It is hoped that Table 1 will provide the practitioner an overview of algorithm capabilities for MOPs and enable them to select the most appropriate approach for their application. Additionally, as we mentioned in Section 8, there is some subjectivity involved in the creation of Table 1; hence, it should be taken as an invitation to our fellow researchers for a debate, which we envisage would have two main outcomes, both of which are very important:

- It will potentially change the relative importance ranks that we have assigned, something that would be beneficial to practitioners.
- It may raise awareness of research areas that deserve more attention.

The idea behind this approach is based in finance, which financial engineers call *price discovery* (Fabozzi, Modigliani, and Ferri 1994). In short, price discovery describes the process by which the market (buyers and sellers) determines the price of *securities* as a result of the *supply* and *demand* mechanism. However, the outlet for such a mechanism is also important as the rankings must be updated as often as possible. For this reason, we are in the process of designing a website to host this mechanism so that it is available to everyone (Giagkiozis 2012).

Seven algorithmic families have been considered as separate entities, although it should be noted that clear boundaries cannot be explicitly drawn. For this reason, researchers start naming algorithms that fall into these categories EAs. This choice is further justified in Section 3. Diffusion of information between the studied methodologies is quite rapid and common. For instance, ESs have come to bear a great resemblance to GAs. Both techniques employ mutation, recombination and selection and the methods for extending GAs to MOPs have successfully been applied in ESs as well as in EDAs, AISs, DE and PSO. Some recombination operators in GAs have similar features to the recombination operator in DE. ACO uses probabilistic transition rules in a similar fashion to EDAs, especially when ACO has continuous decision variables.

It is also evident that more and more researchers lean towards the development of algorithmic processes that exhibit strong adaptive behaviour and provide more information than just the PF approximation at the end of the optimisation procedure. This is partly due to the fact that computational resources have become cheaper and more accessible and partly due to the ever-increasing complexity of systems requiring more elaborate, effective and informative techniques. Additionally, as MOEA research moves toward many objectives, i.e. problems with more than three objectives, research is moving away from Pareto-based algorithms. This is mostly due to the difficulty that is posed in many dimensions for methods employing this

ranking scheme (Purshouse and Fleming 2003; Hughes 2005). However, there are still a number of issues with decomposition-based algorithms. For instance, as the number of dimensions increases, the distance of the weighting vectors effectively increases, and recombination of neighbouring solutions becomes problematic as a result. Due to this problem, many solutions that are potentially Pareto-optimal can be lost or remain unutilised. A suitable solution to this problem has not yet emerged and is a very interesting direction for future research. Several other approaches that seem promising have emerged, for example, cone ε -dominance (Batista et al. 2011) and δ -ball dominance (Chen, Zou, and Xie 2011). However, these methods are quite novel and have not yet been tested for more than three objectives; further investigations are required to reveal their potential merit. Another promising research direction is that of many-objective optimisation in the presence of noise, extending to time-varying problems and real-time optimisation.

Arising from this overview, it becomes apparent that the current trend in population-based multi-objective algorithms is toward estimation of distribution algorithms and indicator-based algorithms. It is the authors' view that a combination of these two methodologies could produce interesting and innovative approaches for many-objective problems. Additionally, more attention from the research community should be ascribed to problems with mixed-type decision variables due to their frequent presence in real-world problems. Another somewhat not fully developed aspect is notation, especially in regard to Pareto-dominance relations, where there is incoherence in this field. Finally, a unified mathematical framework seems possible and its development would be highly beneficial; some steps in this direction can be seen in Laumanns, Zitzler, and Thiele (2000) and Liu et al. (2011).

Acknowledgements

The first author would like to thank Dr S.N. Walker for his insightful comments on this work.

Notes

1. Non-deterministic polynomial time (NP).
2. This is valid in a minimisation context, while the opposite is true in a maximisation context.
3. As a reminder, n represents the number of decision variables.
4. See the next section for a definition of dominance.
5. For a minimisation problem.
6. Referred to as Edgeworth–Pareto dominance relations by some authors.
7. In the case of two objectives.
8. 'Many' meaning more than three in this context.
9. To the authors' best knowledge.
10. According to ISI Web of Knowledge, from 1990 to 2012 there have been published approximately 55 000 papers relating to one or more of the seven algorithm families discussed in this work.
11. To the authors' best knowledge.

Notes on contributors



Ioannis Giagkiozis received his BSc degree from TEI of Thessaloniki, Thessaloniki, Greece, in 2009. He then obtained his MSc degree in control and systems engineering with distinction from the University of Sheffield, Sheffield, United Kingdom, in 2010, for which he was awarded the Nicholson Prize for the most outstanding student. He joined the Department of Automatic Control and Systems Engineering, University of Sheffield, Sheffield, as a research associate in 2011 and in 2013 received his PhD degree in multi-objective non-convex optimisation.

His research interests are in many-objective optimisation, estimation of distribution algorithms and applied convex optimisation.



Robin C. Purshouse Robin Purshouse received the MEng degree in control systems engineering in 1999 and PhD degree in control systems in 2004 for his research on evolutionary many-objective optimisation. Commercial experience includes Logica plc (1999–2000), PA Consulting Group (2003–2007) and Rolls-Royce plc (2007–2008). He returned to academia in 2008 as a research

fellow in the School of Health and Related Research at the University of Sheffield and was appointed as lecturer in the Department of Automatic Control and Systems Engineering in 2010.



Peter J. Fleming has been a professor of industrial systems and control at the University of Sheffield since 1991. From 1993 to 2012, he was the director of the Rolls-Royce University Technology Centre in Control and System Engineering at the University of Sheffield. He was the Head of Automatic Control and Systems Engineering from 1993 to 1999, Director of Research

(Engineering) from 2001 to 2003 and Pro Vice-Chancellor for External Relations from 2003 to 2008. His control and systems engineering research interests include multicriteria decision-making, optimisation, grid computing, and industrial applications of modelling, monitoring and control. He has over 400 research publications, including six books, and his research interests have led to the development of close links with a variety of industries in sectors such as aerospace, power generation, food processing, pharmaceuticals and manufacturing.

Prof. Fleming is a Fellow of the Royal Academy of Engineering, both a Fellow of, and Adviser to, the International Federation of Automatic Control, a Fellow of the Institution of Electrical Engineers, a Fellow of the Institute of Measurement and Control, an Advisor to the International Federation of Automatic Control, and the Editor-in-Chief of the *International Journal of Systems Science*.

References

- Adra, S., Dodd, T., Griffin, I., and Fleming, P. (2009), 'Convergence Acceleration Operator for Multiobjective Optimization', *IEEE Transactions on Evolutionary Computation*, 13(4), 825–847.
- Adra, S., and Fleming, P. (2009), 'A Diversity Management Operator for Evolutionary Many-Objective Optimisation', in *Evolutionary Multi-Criterion Optimization*, New York: Springer, pp. 81–94.

- Akay, B., and Karaboga, D. (2012), 'A Modified Artificial Bee Colony Algorithm for Real-Parameter Optimization', *Information Sciences*, 192, 120–142.
- AlRashidi, M., and El-Hawary, M. (2009), 'A Survey of Particle Swarm Optimization Applications in Electric Power Systems', *IEEE Transactions on Evolutionary Computation*, 13(4), 913–918.
- Bäck, T., Hoffmeister, F., and Schwefel, H.P. (1991), 'A Survey of Evolution Strategies', in *International Conference on Genetic Algorithms*, pp. 2–9.
- Bäck, T., Schütz, M., Ack, T., and Utz, M. (1995), 'Evolution Strategies for Mixed-Integer Optimization of Optical Multilayer Systems', in *Evolutionary Programming IV – Proc. FOURTH Annual Conf. Evolutionary Programming (EP-95)*, Cambridge, MA: MIT Press, pp. 33–51.
- Bader, J. (2009), 'Hypervolume-Based Search for Multiobjective Optimization: Theory and Methods', PhD dissertation, ETH Zurich.
- Bader, J., and Zitzler, E. (2008), 'HypE: An Algorithm for Fast Hypervolume-Based Many-Objective Optimization', Technical Report, Computer Engineering and Networks Laboratory (TIK), ETH Zurich.
- Bader, J., and Zitzler, E. (2011), 'HypE: An Algorithm for Fast Hypervolume-Based Many-Objective Optimization', *IEEE Transactions on Evolutionary Computation*, 19(1), 45–76.
- Baluja, S., and Caruana, R. (1995), 'Removing the Genetics from the Standard Genetic Algorithm', in *Machine Learning International Workshop*, Burlington, MA: Morgan Kaufmann Publishers, pp. 38–46.
- Batista, L., Campelo, F., Guimarães, F., and Ramírez, J. (2011), 'Pareto Cone ϵ -Dominance: Improving Convergence and Diversity in Multiobjective Evolutionary Algorithms', in *Evolutionary Multi-Criterion Optimization*, New York: Springer, pp. 76–90.
- Bell, J., and McMullen, P. (2004), 'Ant Colony Optimization Techniques for the Vehicle Routing Problem', *Advanced Engineering Informatics*, 18(1), 41–48.
- Ben Said, L., Bechikh, S., and Ghédira, K. (2010), 'The r-dominance: A New Dominance Relation for Interactive Evolutionary Multicriteria Decision Making', *IEEE Transactions on Evolutionary Computation*, 14(5), 801–818.
- Bernheim, B. (1984), 'Rationalizable Strategic Behavior', *Econometrica*, 52, 1007–1028.
- Beyer, H. (1997), 'An Alternative Explanation for the Manner in Which Genetic Algorithms Operate', *BioSystems*, 41(1), 1–15.
- Beyer, H., and Schwefel, H. (2002), 'Evolution Strategies - A Comprehensive Introduction', *Natural computing*, 1(1), 3–52.
- Bieniawski, S. (2005), 'Distributed Optimization and Flight Control Using Collectives', PhD dissertation, Stanford University.
- Bieniawski, S., Wolpert, D., and Kroo, I. (2004), 'Discrete, Continuous, and Constrained Optimization Using Collectives', in *Proceedings of the 10th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, pp. 1–9.
- Bilchev, G., and Parmee, I. (1995), 'The Ant Colony Metaphor for Searching Continuous Design Spaces', in *Evolutionary Computing*, New York: Springer, pp. 25–39.
- Bleuler, S., Laumanns, M., Thiele, L., and Zitzler, E. (2003), 'PISA – A Platform and Programming Language Independent Interface for Search Algorithms', in *Evolutionary Multi-Criterion Optimization (EMO 2003)*, New York: Springer, pp. 494–508.
- Bosman, P., and Thierens, D. (2000), 'Expanding from Discrete to Continuous Estimation of Distribution Algorithms: The IDEa', in *Parallel Problem Solving from Nature – PPSN VI*, New York: Springer, pp. 767–776.
- Boyd, S., and Vandenberghe, L. (2004), *Convex Optimization*, Cambridge: Cambridge University Press.
- Brockhoff, D., Friedrich, T., Hebbinghaus, N., Klein, C., Neumann, F., and Zitzler, E. (2007), 'Do Additional Objectives Make a Problem Harder?', in *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, pp. 765–772.
- Bui, L., Abbass, H., Barlow, M., and Bender, A. (2012), 'Robustness Against the Decision-Maker's Attitude to Risk in Problems With Conflicting Objectives', *IEEE Transactions on Evolutionary Computation*, 16(1), 1–19.
- Cai, J., and Thierauf, G. (1996), 'Evolution Strategies for Solving Discrete Optimization Problems', *Advances in Engineering Software*, 25(2–3), 177–183.
- Castro, P., and Von Zuben, F. (2010a), 'GAIS: A Gaussian Artificial Immune System for Continuous Optimization', in *Artificial Immune Systems*, New York: Springer, pp. 171–184.
- Castro, P., and Von Zuben, F. (2010b), 'A Gaussian Artificial Immune System for Multi-Objective Optimization in Continuous Domains', in *Conference on Hybrid Intelligent Systems*, pp. 159–164.
- Chen, X., Ong, Y., Lim, M., and Tan, K. (2011), 'A Multi-Facet Survey on Memetic Computation', *IEEE Transactions on Evolutionary Computation*, 15(5), 591–607.
- Chen, Y., Zou, X., and Xie, W. (2011), 'Convergence of Multi-Objective Evolutionary Algorithms to a Uniformly Distributed Representation of the Pareto Front', *Information Sciences*, 181(16), 3336–3355.
- Chiong, R., and Kirley, M. (2012), 'Effects of Iterated Interactions in Multiplayer Spatial Evolutionary Games', *IEEE Transactions on Evolutionary Computation*, 16(4), 537–555.
- Choe, J., and Perlman, D. (1997), 'Social Conflict and Cooperation Among Founding Queens in Ants (Hymenoptera: Formicidae)', in *The Evolution of Social Behavior in Insects and Arachnids*, Cambridge: Cambridge University Press, pp. 392–406.
- Coello, C., and Cortés, N. (2002), 'An Approach to Solve Multi-objective Optimization Problems Based on an Artificial Immune System', in *First International Conference on Artificial Immune Systems (ICARIS'2002)*, pp. 212–221.
- Coello, C., and Cortés, N. (2005), 'Solving Multiobjective Optimization Problems Using An Artificial Immune System', *Genetic Programming and Evolvable Machines*, 6(2), 163–190.
- Coloni, A., Dorigo, M., and Maniezzo, V. (1991), 'Distributed Optimization by Ant Colonies', in *Proceedings of the First European Conference on Artificial Life (Vol. 142)*, pp. 134–142.
- da Cruz, A., Cardoso, R., Wanner, E., and Takahashi, R. (2011), 'Using Convex Quadratic Approximation as a Local Search Operator in Evolutionary Multiobjective Algorithms', in *2011 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1217–1224.
- Darwin, C. (1858), *The Origin of Species*, Raleigh, NC: Hayes Barton Press.
- Das, I., and Dennis, J. (1996), 'Normal-Boundary Intersection: An Alternate Method for Generating Pareto Optimal Points in Multicriteria Optimization Problems', Technical Report, DTIC Document.
- Das, S., and Suganthan, P. (2010), 'Differential Evolution: A Survey of the State-of-the-Art', *IEEE Transactions on Evolutionary Computation*, 99, 1–28.

- Dasgupta, D., and Attouh-Okine, N. (1997), 'Immunity-Based Systems: A Survey', in *IEEE International Conference on Systems, Man, and Cybernetics*, 1, pp. 369–374.
- Dasgupta, D., and Forrest, S. (1999), 'Artificial Immune Systems in Industrial Applications', in *International Conference on Intelligent Processing and Manufacturing of Materials*, 1, pp. 257–267.
- De Boer, P., Kroese, D., Mannor, S., and Rubinstein, R. (2005), 'A Tutorial on the Cross-Entropy Method', *Annals of Operations Research*, 134(1), 19–67.
- de Lange, R., Samoilovich, I., and van der Rhee, B. (2012), 'Virtual Queuing at Airport Security Lanes', *European Journal of Operational Research*, 225, 153–165.
- Deb, K., and Agrawal, R. (1994), 'Simulated Binary Crossover for Continuous Search Space', *Complex Systems*, 50(9), 115–148.
- Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002), 'A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II', *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197.
- Deb, K., Thiele, L., Laumanns, M., and Zitzler, E. (2002), 'Scalable Multi-Objective Optimization Test Problems', in *Congress on Evolutionary Computation* (Vol. 1), pp. 825–830.
- del Valle, Y., Venayagamoorthy, G., Mohagheghi, S., Hernandez, J., and Harley, R. (2008), 'Particle Swarm Optimization: Basic Concepts, Variants and Applications in Power Systems', *IEEE Transactions on Evolutionary Computation*, 12(2), 171–195.
- Deneubourg, J., Aron, S., Goss, S., and Pasteels, J. (1990), 'The Self-Organizing Exploratory Pattern of the Argentine Ant', *Journal of Insect Behavior*, 3(2), 159–168.
- Dorigo, M., Birattari, M., and Stutzle, T. (2006), 'Ant Colony Optimization', *IEEE Computational Intelligence Magazine*, 1(4), 28–39.
- Dorigo, M., and Blum, C. (2005), 'Ant Colony Optimization Theory: A Survey', *Theoretical Computer Science*, 344(2–3), 243–278.
- Dorigo, M., and Di Caro, G. (1999), 'Ant Colony Optimization: A New Meta-Heuristic', in *Congress on Evolutionary Computation*, Vol. 2.
- Dorigo, M., Maniezzo, V., and Coloni, A. (1991), 'The Ant System: An Autocatalytic Optimizing Process', TR91-016, Politecnico di Milano.
- Dorigo, M., Stützle, T., Farooq, M., Blum, C. (2013), *Ant Colony Optimization - Techniques and Applications*, New York: In-Tech.
- Durillo, J.J., and Nebro, A.J. (2011), 'jMetal: A Java Framework for Multi-Objective Optimization', *Advances in Engineering Software*, 42(10), 760–771.
- Eberhart, R., and Kennedy, J. (1995), 'A New Optimizer Using Particle Swarm Theory', in *International Symposium on Micro Machine and Human Science*, pp. 39–43.
- Edgeworth, F. (1881), *Mathematical Psychics: An Essay on the Application of Mathematics to the Moral Sciences* (10), London: C. Kegan Paul & Company.
- Emmerich, M., Grötzner, M., Groß, B., and Schütz, M. (2000), 'Mixed-Integer Evolution Strategy for Chemical Plant Optimization with Simulators', in *Evolutionary Design and Manufacture-Selected papers from ACDM*, pp. 55–67.
- Fabozzi, F., Modigliani, F., and Ferri, M. (1994), *Foundations of Financial Markets and Institutions* (3), Upper Saddle River, NJ: Prentice Hall.
- Farmer, J., Packard, N., and Perelson, A. (1986), 'The Immune System, Adaptation, and Machine Learning', *Physica D: Non-linear Phenomena*, 22(1–3), 187–204.
- Fonseca, C., and Fleming, P. (1993), 'Genetic Algorithms for Multiobjective Optimization: Formulation, Discussion and Generalization', in *Conference on Genetic Algorithms* (Vol. 423), pp. 416–423.
- Fonseca, C., and Fleming, P. (1995), 'Multiobjective Genetic Algorithms Made Easy: Selection Sharing and Mating Restriction', in *International Conference on Genetic Algorithms in Engineering Systems: Innovations and Applications*, pp. 45–52.
- Fourman, M. (1985), 'Compaction of Symbolic Layout Using Genetic Algorithms', in *Proceedings of the 1st International Conference on Genetic Algorithms*, pp. 141–153.
- Gagné, C., and Parizeau, M. (2006), 'Genericity in Evolutionary Computation Software Tools: Principles and Case Study', *International Journal on Artificial Intelligence Tools*, 15(2), 173–194.
- Gaing, Z. (2005), 'Constrained Optimal Power Flow by Mixed-Integer Particle Swarm Optimization', in *Power Engineering Society General Meeting*, pp. 243–250.
- Gasper, A., and Collard, P. (1999), 'From GAs to Artificial Immune Systems: Improving Adaptation in Time Dependent Optimization', in *Proceedings of the Congress on Evolutionary Computation*, pp. 1859–1866.
- Giagkiozis, I. (2012), <http://ioannis-giagkiozis.staff.shef.ac.uk/>.
- Giagkiozis, I., and Fleming, P. (2012), 'Methods for Many-Objective Optimization: An Analysis', ACSE Research Report no. 1030, Automatic Control and Systems Engineering, University of Sheffield.
- Giagkiozis, I., Purshouse, R., and Fleming, P. (2012), 'Generalized Decomposition and Cross Entropy Methods for Many-Objective Optimization', Research Report No. 1029, Department of Automatic Control and Systems Engineering, The University of Sheffield.
- Giagkiozis, I., Purshouse, R., and Fleming, P. (2013), 'Generalized Decomposition', in *Evolutionary Multi-Criterion Optimization*, Lecture Notes in Computer Science, Springer Berlin.
- Goldberg, D. (1989), 'Genetic Algorithms in Search, Optimization, and Machine Learning'.
- Goldberg, D., and Holland, J. (1988), 'Genetic Algorithms and Machine Learning', *Machine Learning*, 3(2), 95–99.
- Gondzio, J. (2012), 'Interior Point Methods 25 years Later', *European Journal of Operational Research*, 218(3), 587–601.
- Gu, F., Liu, H., and Tan, K. (2012), 'A Multiobjective Evolutionary Algorithm Using Dynamic Weight Method', *International Journal of Innovative Computing, Information and Control*, 8(5B), 3677–3688.
- Hadka, D., and Reed, P. (2012), 'Diagnostic Assessment of Search Controls and Failure Modes in Many - Objective Evolutionary Optimization', *Evolutionary Computation*, 20(3), 423–452.
- Halmos, P. (1974), *Measure Theory* (18), New York: Springer.
- Hauschild, M., and Pelikan, M. (2011), 'A Survey of Estimation of Distribution Algorithms', MEDAL Report No. 2011004, Missouri Estimation of Distribution Algorithms Laboratory.
- He, S., Wu, Q., and Saunders, J. (2009), 'Group Search Optimizer: An Optimization Algorithm Inspired by Animal Searching Behavior', *IEEE Transactions on Evolutionary Computation*, 13(5), 973–990.
- Hertz, A., and Widmer, M. (2003), 'Guidelines for the use of Meta-Heuristics in Combinatorial Optimization', *European Journal of Operational Research*, 151(2), 247–252.
- Holland, J. (1962), 'Outline for a Logical Theory of Adaptive Systems', *Journal of the ACM (JACM)*, 9(3), 297–314.
- Holland, J. (1975), *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology*,

- Control, and Artificial Intelligence*, Cambridge, MA: A Bradford Book.
- Huband, S., Hingston, P., Barone, L., and While, L. (2006), 'A Review of Multiobjective Test Problems and A Scalable Test Problem Toolkit', *IEEE Transactions on Evolutionary Computation*, 10(5), 477–506.
- Hughes, E.J. (2005), 'Evolutionary Many-Objective Optimisation: Many Once or One Many?', *IEEE Congress on Evolutionary Computation*, 1, 222–227.
- Igel, C., Heidrich-Meisner, V., and Glasmachers, T. (2008), 'Shark', *Journal of Machine Learning Research*, 9, 993–996.
- Ishibuchi, H., Tsukamoto, N., and Nojima, Y. (2008), 'Evolutionary Many-Objective Optimization: A Short Review', in *IEEE Congress on Evolutionary Computation*, pp. 2419–2426.
- Jaszkiewicz, A. (2002), 'On the Performance of Multiple-Objective Genetic Local Search on the 0/1 Knapsack Problem - A comparative Experiment', *IEEE Transactions on Evolutionary Computation*, 6(4), 402–412.
- Jaszkiewicz, A., and Dabrowski, G. (2012), *MOMH Multiple-Objective MetaHeuristics*, <http://home.gna.org/momh/index.html>.
- Jiang, S., Cai, Z., Zhang, J., and Ong, Y.S. (2011a), 'Multiobjective Optimization by Decomposition with Pareto-Adaptive Weight Vectors', *International Conference on Natural Computation*, 3, 1260–1264.
- Jiang, S., Zhang, J., and Ong, Y. (2011b), *Asymmetric Pareto-Adaptive Scheme for Multiobjective Optimization* (7106), Berlin, Heidelberg: Springer.
- Jin, Y., and Branke, J. (2005), 'Evolutionary Optimization in Uncertain Environments - A Survey', *IEEE Transactions on Evolutionary Computation*, 9(3), 303–317.
- Jin, Y., Olhofer, M., and Sendhoff, B. (2001), 'Dynamic Weighted Aggregation for Evolutionary Multi-Objective Optimization: Why Does it Work and How?', in *Proceedings of the Genetic and Evolutionary Computation Conference GECCO*, pp. 1042–1049.
- Jin, Y., Olhofer, M., and Sendhoff, B. (2002), 'A Framework for Evolutionary Optimization with Approximate Fitness Functions', *IEEE Transactions on Evolutionary Computation*, 6(5), 481–494.
- Karaboga, D., Ozturk, C., Karaboga, N., and Gorkemli, B. (2012), 'Artificial Bee Colony Programming for Symbolic Regression', *Information Sciences*, 209, 1–15.
- Keijzer, M., Merelo, J., Romero, G., and Schoenauer, M. (2002), *Evolving Objects: A General Purpose Evolutionary Computation Library* (Vol. 2310), Berlin, Heidelberg: Springer.
- Kennedy, J., and Eberhart, R. (1995), 'Particle Swarm Optimization', *IEEE International Conference on Neural Networks*, 4, 1942–1948.
- Kennedy, J., and Eberhart, R. (1997), 'A Discrete Binary Version of the Particle Swarm Algorithm', *IEEE International Conference on Systems, Man, and Cybernetics*, 5, 4104–4108.
- Kim, J., Han, J., Kim, Y., Choi, S., and Kim, E. (2012), 'Preference-Based Solution Selection Algorithm for Evolutionary Multi-objective Optimization', *IEEE Transactions on Evolutionary Computation*, 16(1), 20–34.
- Koza, J. (1996), *On the Programming of Computers by Means of Natural Selection* (1), Cambridge, MA: MIT Press.
- Krammer, P. (2000), 'CD95's Deadly Mission in the Immune System', *Nature*, 407, 789–795.
- Kronfeld, M., Planatscher, H., and Zell, A. (2010), 'The EvA2 Optimization Framework', in *Learning and Intelligent Optimization Conference, Special Session on Software for Optimization (LION-SWOP)*, eds. C. Blum and R. Battiti, Venice, Italy: Springer-Verlag, pp. 247–250.
- Kursawe, F. (1991), 'A Variant of Evolution Strategies for Vector Optimization', *Parallel Problem Solving from Nature*, 496, 193–197.
- Lampinen, J., and Zelinka, I. (1999), 'Mixed Integer-Discrete-Continuous Optimization by Differential Evolution', in *Proceedings of the 5th International Conference on Soft Computing*, pp. 71–76.
- Larranaga, P., and Lozano, J. (2002), *Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation* (2), The Netherlands: Springer.
- Laumanns, M., and Ocenasek, J. (2002), 'Bayesian Optimization Algorithms for Multi-Objective Optimization', in *Parallel Problem Solving from Nature – PPSN VII*, New York: Springer, pp. 298–307.
- Laumanns, M., Thiele, L., Deb, K., and Zitzler, E. (2002), 'Combining Convergence and Diversity in Evolutionary Multi-objective Optimization', *Evolutionary Computation*, 10(3), 263–282.
- Laumanns, M., Zitzler, E., and Thiele, L. (2000), 'A Unified Model for Multi-Objective Evolutionary Algorithms with Elitism', *IEEE Congress on Evolutionary Computation*, 1, 46–53.
- Li, H., and Zhang, Q. (2009), 'Multiobjective Optimization Problems with Complicated Pareto Sets, MOEA/D and NSGA-II', *IEEE Transactions on Evolutionary Computation*, 13(2), 284–302.
- Liefvooghe, A., Basseur, M., Jourdan, L., and Talbi, E.G. (2007), 'ParadisEO-MOEO: A Framework for Evolutionary Multi-Objective Optimization', in *Evolutionary Multi-criterion Optimization*, New York: Springer, pp. 386–400.
- Lim, D., Jin, Y., Ong, Y., and Sendhoff, B. (2010), 'Generalizing Surrogate-Assisted Evolutionary Computation', *IEEE Transactions on Evolutionary Computation*, 14(3), 329–355.
- Liu, B., Wang, L., Liu, Y., and Wang, S. (2011), 'A Unified Framework for Population-Based Metaheuristics', *Annals of Operations Research*, 186, 231–262.
- López-Ibáñez, M., and Stützle, T. (2010), 'An Analysis of Algorithmic Components for Multiobjective Ant Colony Optimization: A Case Study on the Biobjective TSP', *Artificial Evolution*, 5975, 134–145.
- Lukasiewicz, M., Glaß, M., Reimann, F., and Teich, J. (2011), 'Opt4J: A Modular Framework for Meta-Heuristic Optimization', in *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation*, pp. 1723–1730.
- Meffert, K., and Rotstan, N. (2010), 'JGAP: Java Genetic Algorithms Package'. <http://jgap.sourceforge.net/>.
- Michalewicz, Z. (2012), 'Quo Vadis, Evolutionary Computation?', *Advances in Computational Intelligence*, 7311, 98–121.
- Miettinen, K. (1999), *Nonlinear Multiobjective Optimization* (vol. 12), New York: Springer.
- Miettinen, K., and Mäkelä, M. (2002), 'On Scalarizing Functions in Multiobjective Optimization', *OR Spectrum*, 24(2), 193–213.
- Mohler, R., Bruni, C., and Gandolfi, A. (1980), 'A Systems Approach to Immunology', *Proceedings of the IEEE*, 68(8), 964–990.
- Moscato, P. (1989), 'On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms', C3P Report 826, California Institute of Technology.
- Muhlenbein, H., and Mahnig, T. (2002), 'Evolutionary Optimization and the Estimation of Search Distributions with Applications to Graph Bipartitioning', *International Journal of Approximate Reasoning*, 31(3), 157–192.
- Mühlenbein, H., and Paass, G. (1996), 'From Recombination of Genes to the Estimation of Distributions I. Binary

- Parameters', in *Parallel Problem Solving from Nature*, New York: Springer, pp. 178–187.
- Ocenasek, J., and Schwarz, J. (2002), 'Estimation Distribution Algorithm for Mixed Continuous-Discrete Optimization Problems', *Intelligent Technologies: Theory and Applications: New Trends in Intelligent Technologies*, 76, 227.
- Ong, Y.S., Nair, P., and Lum, K. (2006), 'Max-Min Surrogate-Assisted Evolutionary Algorithm for Robust Design', *IEEE Transactions on Evolutionary Computation*, 10(4), 392–404.
- Ono, I., and Kobayashi, S. (1997), 'A Real-Coded Genetic Algorithm for Function Optimization Using Unimodal Normal Distribution Crossover', *Journal of Japanese Society for Artificial Intelligence*, 14(6), 246–253.
- Pampara, G., Engelbrecht, A., and Franken, N. (2006), 'Binary Differential Evolution', in *IEEE Congress on Evolutionary Computation*, pp. 1873–1879.
- Pan, Q., Tasgetiren, M., and Liang, Y. (2007), 'A Discrete Differential Evolution Algorithm for the Permutation Flowshop Scheduling Problem', in *Conference on Genetic and Evolutionary Computation*, pp. 126–133.
- Panait, L., Balan, G., Paus, S., Skolicki, Z., Popovici, E., Sullivan, K., Harrison, J., Bassett, J., Hubley, R., Chircop, A. (2010), 'ECJ: A Java-Based Evolutionary Computation Research System', <http://code.google.com/p/ecj/>.
- Pareto, V. (1896), *Cours D'Économie Politique*, Encyclopaedia Britannica.
- Parham, P., and Janeway, C. (2005), *The Immune System*, New York: Garland Science.
- Parijs, L., and Abbas, A. (1998), 'Homeostasis and Self-Tolerance in the Immune System: Turning Lymphocytes Off', *Science*, 280(5361), 243.
- Parsopoulos, K., and Vrahatis, M. (2002), 'Particle Swarm Optimization Method in Multiobjective Problems', in *Proceedings of the 2002 ACM Symposium on Applied Computing*, pp. 603–607.
- Pelikan, M., Goldberg, D., and Lobo, F. (2002), 'A Survey of Optimization by Building and Using Probabilistic Models', *Computational Optimization and Applications*, 21(1), 5–20.
- Pelikan, M., Goldberg, D., and Tsutsui, S. (2005), 'Hierarchical Bayesian Optimization Algorithm: Toward A New Generation of Evolutionary Algorithms', *SICE Annual Conference*, 3, 2738–2743.
- Pelikan, M., and Sastry, K. (2006), *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications* (vol. 33), Berlin, Heidelberg: Springer-Verlag.
- Poloni, C., Giurgevich, A., Onesti, L., and Pediroda, V. (2000), 'Hybridization of A Multi-Objective Genetic Algorithm, A Neural Network and A Classical Optimizer for A Complex Design Problem in Fluid Dynamics', *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 403–420.
- Puchinger, J., and Raidl, G.R. (2005), 'Combining Metaheuristics and Exact Algorithms in Combinatorial Optimization: A Survey and Classification', *Lecture Notes in Computer Science*, 3562, 41–53.
- Purshouse, R.C., and Fleming, P.J. (2003), 'Conflict, Harmony, and Independence: Relationships in Evolutionary Multi-Criterion Optimisation', in *Conference on Evolutionary Multi-Criterion Optimization*, pp. 16–30.
- Purshouse, R., and Fleming, P. (2003), 'Evolutionary Many-Objective Optimisation: An Exploratory Analysis', *IEEE Congress on Evolutionary Computation*, 3, 2066–2073.
- Purshouse, R., and Fleming, P. (2007), 'On the Evolutionary Optimization of Many Conflicting Objectives', *IEEE Transactions on Evolutionary Computation*, 11(6), 770–784.
- Rechenberg, I. (1965), *Cybernetic Solution Path of An Experimental Problem*, Germany: Royal Aircraft Establishment Library Translation.
- Reyes-Sierra, M., and Coello, C. (2006), 'Multi-Objective Particle Swarm Optimizers: A Survey of the State-of-the-Art', *International Journal of Computational Intelligence Research*, 2(3), 287–308.
- Rubinstein, R. (1999), 'The Cross-Entropy Method for Combinatorial and Continuous Optimization', *Methodology and Computing in Applied Probability*, 1(2), 127–190.
- Rubinstein, R. (2005), 'A Stochastic Minimum Cross-Entropy Method for Combinatorial Optimization and Rare-event Estimation', *Methodology and Computing in Applied Probability*, 7(1), 5–50.
- Sastry, K. (2001), 'Efficient Cluster Optimization Using Extended Compact Genetic Algorithm with Seeded Population', in *Proceedings of the Optimization by Building and Using Probabilistic Models OBUPM Workshop at the Genetic and Evolutionary Computation Conference (GECCO-2001 OBUPM)*.
- Sastry, K., Goldberg, D., and Pelikan, M. (2005), 'Limits of Scalability of Multiobjective Estimation of Distribution Algorithms', *IEEE Congress on Evolutionary Computation*, 3, 2217–2224.
- Saxena, D., Zhang, Q., Duro, J., and Tiwari, A. (2011), 'Framework for Many-Objective Test Problems With Both Simple and Complicated Pareto-Set Shapes', in *Evolutionary Multi-Criterion Optimization*, New York: Springer, pp. 197–211.
- Schaffer, J. (1985), 'Multiple Objective Optimization with Vector Evaluated Genetic Algorithms', in *Conference on Genetic Algorithms*, pp. 93–100.
- Scharnow, J., Tinnefeld, K., and Wegener, I. (2004), 'The Analysis of Evolutionary Algorithms on Sorting and Shortest Paths Problems', *Journal of Mathematical Modelling and Algorithms*, 3(4), 349–366.
- Schwefel, H. (1975), *Evolutionsstrategie und Numerische Optimierung*, Berlin: Technische Universität Berlin.
- Schwefel, H. (1981), *Numerical Optimization of Computer Models*, New York: Wiley.
- SCOPUS (2012), <http://www.scopus.com/>.
- Shah, R., and Reed, P. (2011), 'Comparative Analysis of Multiobjective Evolutionary Algorithms for Random and Correlated Instances of Multiobjective D-Dimensional Knapsack Problems', *European Journal of Operational Research*, 211(3), 466–479.
- Shi, Y., and Eberhart, R. (1998), 'A Modified Particle Swarm Optimizer', in *IEEE International Conference on Evolutionary Computation*, pp. 69–73.
- Socha, K. (2004), 'ACO for Continuous and Mixed-Variable Optimization', in *Ant Colony Optimization and Swarm Intelligence*, New York: Springer, pp. 53–61.
- Socha, K., and Dorigo, M. (2008), 'Ant Colony Optimization for Continuous Domains', *European Journal of Operational Research*, 185(3), 1155–1173.
- Srinivas, N., and Deb, K. (1994), 'Multiobjective Optimization Using Nondominated Sorting in Genetic Algorithms', *Evolutionary Computation*, 2(3), 221–248.
- Storn, R. (1996), 'On the Usage of Differential Evolution for Function Optimization', in *Conference of the North American Fuzzy Information Processing Society*, pp. 519–523.
- Storn, R., and Price, K. (1995), *Differential Evolution - A Simple and Efficient Adaptive Scheme for Global Optimization Over Continuous Spaces*, Berkeley, CA: International Computer Science Institute-Publications TR.
- Tan, Y.Y., Jiao, Y.C., Li, H., and Wang, X.K. (2012), 'MOEA/D + Uniform Design: A New Version of MOEA/D for

- Optimization Problems With Many Objectives', *Computers & Operations Research*, 40, 1648–1660.
- Tapia, M., and Coello, C. (2007), 'Applications of Multi-Objective Evolutionary Algorithms in Economics and Finance: A Survey', in *IEEE Congress on Evolutionary Computation*, pp. 532–539.
- Teytaud, O. (2006), 'How Entropy-Theorems can Show that Approximating High-Dim Pareto-fronts is too Hard', in *Bridging the Gap between Theory and Practice-Workshop PPSN-BTP*.
- Thierens, D., and Bosman, P. (2001), 'Multi-Objective Mixture-Based Iterated Density Estimation Evolutionary Algorithms', in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 663–670.
- Torosl, I.H., and Arslanoglu, Y. (2007), 'Genetic Algorithm for the Personnel Assignment Problem With Multiple Objectives', *Information Sciences*, 177(3), 787–803.
- Tripathi, P., Bandyopadhyay, S., and Pal, S. (2007), 'Multi-Objective Particle Swarm Optimization with Time Variant Inertia and Acceleration Coefficients', *Information Sciences*, 177(22), 5033–5049.
- Ullman, J. (2009), 'Advising Students for Success', *Communications of the ACM*, 52(3), 34–37.
- Van Veldhuizen, D. (1999), 'Multiobjective Evolutionary Algorithms: Classifications, Analyses, and New Innovations', PhD dissertation, Air Force Institute of Technology Wright Patterson AFB, OH, USA.
- Voigt, H., Mühlenbein, H., and Cvetkovic, D. (1995), 'Fuzzy Recombination for the Breeder Genetic Algorithm', in *Proceedings of Sixth International Conference on Genetic Algorithms*, pp. 104–111.
- Wagner, S., Beham, A., Kronberger, G., Kommenda, M., Pitzer, E., Kofler, M., Vonolfen, S., Winkler, S., Dorfer, V., and Afenzeller, M. (2010), 'Heuristicslab 3.3: A Unified Approach to Metaheuristic Optimization', in *Actas del séptimo congreso español sobre Metaheurísticas, Algoritmos Evolutivos y Bioinspirados (MAEB'2010)*, p. 8.
- Wagner, T., Beume, N., and Naujoks, B. (2007), 'Pareto-, Aggregation-, and Indicator-Based Methods in Many-Objective Optimization', in *Evolutionary Multi-Criterion Optimization*, New York: Springer, pp. 742–756.
- Wang, R., Purshouse, R., and Fleming, P. (2012), 'Preference-Inspired Co-Evolutionary Algorithms for Many-objective Optimisation', *IEEE Transactions on Evolutionary Computation*, PP(99), 1.
- Wanner, E., Guimarães, F., Takahashi, R., and Fleming, P. (2008), 'Local Search With Quadratic Approximations into Memetic Algorithms for Optimization With Multiple Criteria', *Evolutionary computation*, 16(2), 185–224.
- Wei, L., Oon, W., Zhu, W., and Lim, A. (2012), 'A Reference Length Approach for the 3D Strip Packing Problem', *European Journal of Operational Research*, 220(1), 37–47.
- Wolpert, D., and Macready, W. (1997), 'No Free Lunch Theorems for Optimization', *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.
- Wolpert, D., Strauss, C., and Rajnarayan, D. (2006), 'Advances in Distributed Optimization Using Probability Collectives', *Advances in Complex Systems*, 9(4), 383–436.
- Yoo, J., and Hajela, P. (1999), 'Immune Network Simulations in Multicriterion Design', *Structural and Multidisciplinary Optimization*, 18(2), 85–94.
- Zhang, Q., and Li, H. (2007), 'MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition', *IEEE Transactions on Evolutionary Computation*, 11(6), 712–731.
- Zhang, Q., Liu, W., and Li, H. (2009), 'The Performance of a New Version of MOEA/D on CEC09 Unconstrained MOP Test Instances', in *IEEE Congress on Evolutionary Computation*, pp. 203–208.
- Zhang, Q., Zhou, A., and Jin, Y. (2008), 'RM-MEDA: A Regularity Model-Based Multiobjective Estimation of Distribution Algorithm', *IEEE Transactions on Evolutionary Computation*, 12(1), 41–63.
- Zhou, A., Qu, B., Li, H., Zhao, S., Suganthan, P., and Zhang, Q. (2011), 'Multiobjective Evolutionary Algorithms: A Survey of the State-of-the-Art', *Swarm and Evolutionary Computation*, 1, 32–49.
- Zitzler, E., Deb, K., and Thiele, L. (2000), 'Comparison of Multi-objective Evolutionary Algorithms: Empirical Results', *Evolutionary Computation*, 8(2), 173–195.
- Zitzler, E., and Künzli, S. (2004), 'Indicator-Based Selection in Multiobjective Search', in *Parallel Problem Solving from Nature*, New York: Springer, pp. 832–842.
- Zitzler, E., Laumanns, M., and Thiele, L. (2001), 'SPEA2: Improving the Strength Pareto Evolutionary Algorithm', *EUROGEN*, 103, 1–21.
- Zitzler, E., and Thiele, L. (1999), 'Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach', *IEEE Transactions on Evolutionary Computation*, 3(4), 257–271.
- Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C., and da Fonseca, V. (2003), 'Performance Assessment of Multiobjective Optimizers: An Analysis and Review', *IEEE Transactions on Evolutionary Computation*, 7(2), 117–132.
- Zlochin, M., Birattari, M., Meuleau, N., and Dorigo, M. (2004), 'Model-Based Search for Combinatorial Optimization: A Critical Survey', *Annals of Operations Research*, 131(1), 373–395.

Appendix A. Web of Knowledge queries

Following are the queries used to obtain [Figures 14](#) and [15](#).

Genetic Algorithms

```
("genetic" AND ("algorithm" OR "algorithms") AND ("optimization" OR "optimisation"))
```

Evolution Strategies

```
("evolution" AND ("strategy" OR "strategies") AND ("optimization" OR "optimisation"))
```

Artificial Immune Systems

```
("artificial immune" AND ("system" OR "systems") AND ("optimization" OR "optimisation"))
```

Ant Colony Optimization

```
("ant colony" OR "ant system") AND ("optimization" OR "optimisation"))
```

Differential Evolution

```
("differential evolution") AND ("optimization" OR "optimisation"))
```

Particle Swarm Optimization

("particle swarm" AND ("optimization" OR "optimisation"))

For every algorithm family we made the search more specific linking the above queries and the following queries with an AND.

Continuous Problems

("continuous" OR "real"
OR "WFG" OR "DTLZ" OR "ZDT")

Discrete Problems

("discrete" OR "binary")

Combinatorial Problems

("combinatorial" OR "travelling salesman" OR "traveling salesman" OR "vehicle routing" OR "knapsack problem" OR "assignment problem" OR "scheduling problem" OR "city courier problem" OR "land use planning")

Multi-Objective Problems

("multi objective" OR "many objective"
OR "multiobjective" OR "manyobjective"
OR "bi criterion" OR "multicriterion"
OR "multi criterion" OR "multiple criteria")